HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach

HOANG XUAN QUY

quy.hx215749@sis.hust.edu.vn

NGUYEN SY HOANG

hoang.ns215709@sis.hust.edu.vn

Advanced Mechatronics Program

Supervisor : PhD. Hoang Sy Tuan

Reviewer : Assoc. Prof. PhD. Pham Duc An

Department : Mechatronics

School : School of Mechanical Engineering

HANOI, FEBRUARY 2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF MECHANICAL ENGINEERING

SOCIALIST REPUBLIC OF VIETNAM
Independence – Freedom - Happiness

GRADUATION THESIS ASSIGNMENT

*(*Major: Mechatronics*)*

I. Student Information

Hoang Xuan Quy – ID: 20215749 Class: ME-E1 02– K66

Nguyen Sy Hoang – ID: 20215709 Class: ME-E1 02– K66

II. Graduation Thesis

Project name: Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach

Project Requirementsu:

- Build the mathematical model of the robot
- Develop simulation and calculation programs using MATLAB software
- Simulate the robot model in Gazebo
- Design the mechanical structure of the robot
- Design the installation and wiring diagrams
- Program the robot on an RTOS platform

III. Task allocation

Hoang Xuan Quy

- Design and assemble the mechanical system of the robot
- Develop the algorithm flowchart
- Implement the algorithms
- Conduct testing and validation

Nguyen Sy Hoang

- Develop the mathematical model

- Set up simulations using MATLAB software
- Visualize the simulation in Gazebo
- Conduct testing and validation

Hanoi, September 2025

Supervisor

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF MECHANICAL ENGINEERING

SOCIALIST REPUBLIC OF VIETNAM
Independence – Freedom - Happiness

EVALUATION OF GRADUATION THESIS

(for Supervisor)

Topic: *Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach*

Full Name	**Class**	**Student ID**
Hoang Xuan Quy	ME-E1 02 - K66	20215749
Nguyen Sy Hoang	ME-E1 02 - K66	20215709

Major: Mechatronics (Elitech)

Supervisor: **Dr. Hoang Sy Tuan**

EVALUATED ASPECTS

I. Working attitude

Throughout the project, the student group consistently demonstrated a serious, proactive, and positive working attitude. With a strong sense of responsibility, well-structured planning, and effective teamwork, they successfully accomplished a substantial amount of work and met all project requirements to a high standard.

II. Achieved results

The thesis achieved the following results:

- Successfully designed and fabricated a physical prototype of a two-wheeled, two-legged robot featuring a multi-joint leg mechanism, enabling the robot to actively adjust its center-of-mass height to adapt to different operating postures.
- Developed a complete mathematical model and established detailed state-space equations to describe the complex dynamic characteristics of the robotic system.
- Designed an adaptive LQR-Fuzzy controller, employing a fuzzy supervisory mechanism to automatically update the optimal parameters of the LQR controller based on the robot leg height state.

- Conducted thorough validation simulations in both MATLAB/Simulink and ROS/Gazebo environments to evaluate the stability of the control algorithm prior to hardware implementation.
- Integrated the electrical and embedded system, including the use of an ESP32 microcontroller for fast signal processing and the application of a Kalman filter to accurately reduce noise in the tilt angle sensor measurements.
- Designed a computer-based monitoring interface that enables response plotting and real-time tracking of the robot's operating status.
- Achieved promising experimental results in controlling the robot prototype, particularly in its self-balancing capability.

III. Limitations

The robot is only capable of maintaining self-balance in place and adjusting its center of gravity vertically. Key functionalities such as autonomous locomotion, obstacle avoidance, mapping (SLAM), and more complex motion skills have not yet been practically implemented.

IV. Conclusion

The supervisor recommends allowing the students to present and defend their graduation thesis in front of the Graduation Thesis Defense Committee.

Evaluation: 10/10 points

Hanoi, Feb. 2026
Supervisor

PhD. Hoang Sy Tuan

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF MECHANICAL ENGINEERING

SOCIALIST REPUBLIC OF VIETNAM
Independence – Freedom - Happiness

FEEDBACK ON GRADUATION THESIS

(For Reviewer)

Project name: Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach

Full name of student:

Hoang Xuan Quy ID: 20215749 Class: ME-E1 02 K66

Nguyen Sy Hoang ID: 20215709 Class: ME-E1 02 K66

Major: Mechatronics (Elitech)

REVIEWER'S REVIEW CONTENT

I. Achievements

……

……

II. Limitations

……

……

……

……

III. Conclusion

The reviewer offered to allow the student to defend the graduation thesis before the Graduation Thesis Committee.

Rating: …… point

Hanoi, Feb. 2026

Reviewer

Assoc. Prof. PhD. Pham Duc An

Acknowledgement

We would like to express our deepest gratitude to the faculty members of the Faculty of Mechatronics at the University of Mechanical Engineering in general, and the lecturers of our department in particular. You have provided us with essential knowledge, helped us overcome numerous challenges, and granted access to laboratory facilities throughout our research process.

In particular, we would like to sincerely thank our supervisor, PhD. Hoang Sy Tuan, for his dedicated guidance in selecting the research topic and his constant support during the implementation phase.

We would also like to thank our seniors for their valuable guidance and for recommending the reference materials used in this research.

However, as this is our first time conducting a project titled “Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach” and working with advanced control algorithms, certain limitations and errors are unavoidable. We look forward to receiving sympathetic feedback, suggestions, and further guidance from our teachers, seniors, and peers.

Finally, we wish our teachers great health, continued success in your careers and lives, and that you may continue to inspire and train talented students for the university and the nation. We also wish our seniors and fellow students health and academic success as you prepare a solid foundation for your future.

Sincerely,

Abstract

The project titled “Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach” is developed based on the inverted pendulum model theory. After comparing and analyzing the advantages and disadvantages of various controllers suitable for a Wheeled Bipedal Robot, the team decided to implement the Linear Quadratic Regulator (LQR) algorithm for control, conducting simulations in both MATLAB and GAZEBO. This approach serves to research the LQR algorithm and its practical application in controlling Wheeled Bipedal Robot systems.

In this research, the team compares the performance of the LQR controller across three specific robot states: low, medium, and high positions. A Fuzzy Supervisor is then utilized to determine the optimal LQR controller parameters for any arbitrary state. Through this method, the Wheeled Bipedal Robot control system achieves the highest possible stability.

The thesis also details the comprehensive system design. The hardware section covers the selection of appropriate components and 3D modeling using SolidWorks. The software section includes the control algorithms and system flowcharts, focusing on LQR control for the motors, sensor signal processing, and the design of a system monitoring interface.

CONTENTS

LIST OF ABBREVIATIONS.....I

CHAPTER 1: INTRODUCTION..... 1

1.1 Problem Statement..... 1

1.1.1 Advantages and Disadvantages of Wheeled and Legged Robots 1

1.1.2 The Hybrid Solution: Variable-Leg-Length Wheel-Legged Robot 2

1.1.3 Control challenges: Why LQR-Fuzzy? 3

1.2 Project Objectives 4

1.3 Project Limitations..... 4

1.4 Research Methodology 4

1.5 Project Contents..... 5

CHAPTER 2: THEORETICAL BACKGROUND..... 6

2.1 Dynamic characteristics..... 6

2.2 Equivalent center of gravity calculation 8

2.3 The inverted pendulum problem..... 9

2.3.1 Parameters of the inverted pendulum system..... 9

2.3.2 Mathematical model of the inverted pendulum system 10

2.3.3 Establish the system of equations..... 11

2.4 Some control methods that can be used for the system 12

2.4.1 Proportional Integral Derivative controller (PID controller) 12

2.4.2 Proportional Integral Derivative controller combined with Genetic Algorithm 13

2.4.3 Fuzzy controller..... 13

2.4.4 Optimal controller 15

2.5 LQR controller design 16

2.5.1 Modeling the inverted pendulum system by mathematical equations 16

2.5.2 Theoretical Background 16

2.5.3 Design of the LQR controller 16

CHAPTER 3: DESIGN AND SIMULATION 19

3.1 Hardware design 19

3.1.1 Design requirements 19

3.1.2 Hardware device selection 19

3.1.3 Power supply block 28

3.1.4 Vehicle frame 30

3.2 SIMULATION 31

3.2.1 MATLAB 31

3.2.2 ROS/GAZEBO 35

CHAPTER 4: ACTUAL MODEL DESIGN 37

4.1 Design requirements 37

4.2 Building the control algorithm 37

4.2.1 Main program algorithm 38

4.2.2 Algorithm for reading and processing values from the tilt angle sensor 42

4.2.3 PWM pulse width modulation algorithm for each wheel using LQR and motor control 43

4.2.4 Output and plotting charts on the computer 44

4.3 Monitoring Software 45

CHAPTER 5: CONCLUSION AND FUTURE PERSPECTIVES 47

5.1 Achieved Results 47

5.1.1 Model Design and Simulation Results 47

5.1.2 Experimental Results 50

5.2 Conclusion 54

5.3 Limitations of the Project 55

5.4 Future Development 55

REFERENCES 57

APPENDICES 59

Appendix 1: Arduino Source Code for the Two-Wheeled Self-Balancing Robot System 59

Appendix 2: MATLAB Script for Center of Mass (CoM) Calculation of the Self-Balancing Robot Body 76

Appendix 3: MATLAB Script for Calculating the LQR Gain Matrix (K) for the Self-Balancing Robot System 78

LIST OF TABLES

Table 2.1. Symbols and descriptions of the system variables..... 7
Table 2.2 D-H PARAMETERS OF THE TLWR 8
Table 3.1. Pin connection table for the MPU6050 sensor 23
Table 3.2. Pin connection table for the BTS7960 DC Motor Driver. 25
Table 3.3. Pin connection table for the DC motors and Encoders. 26
Table 5.1. Parameters of the two-legged wheeled robot..... 47
Table 5.2. Parameters of the two-legged wheeled robot..... 48

LIST OF FIGURES

Figure 1.1 Wheel-Legged Robot navigates various challenging terrains. 3
Figure 1.2. Wheel-Legged Robot for patrolling and security surveillance..... 3
Figure 2.1. Diagram of different types of robot dynamics models. 6
Figure 2.2. Kinematic parameters of the TLWR (Two-Wheeled Legged Robot). 8
Figure 2.3. Mathematical model of the inverted pendulum on a vehicle system. . 9
Figure 2.4. Basic structure of a Fuzzy Logic Controller..... 14
Figure 2.5. Modeling of the inverted pendulum on a cart system in Simulink.... 16
Figure 2.6. Block diagram of the Linear Quadratic Regulator (LQR) controller. 17
Figure 3.1. Pinout diagram of the ESP32 microcontroller..... 21
Figure 3.2. MPU6050 IMU tilt sensor. 22
Figure 3.3. BTS7960 DC Motor Driver Module – 43A High-power Motor Driver 24
Figure 3.4. GM25-370 DC Geared Motor with Encoder..... 26
Figure 3.5. TD-8120MG 270-degree Digital RC Servo Motor. 27
Figure 3.6. Components of the Power Supply Module..... 29
Figure 3.7. Constructing the robot frame using SolidWorks software. 31
Figure 3.8. Three distinct postures of the TLWR (Two-Wheeled Legged Robot). 31
Figure 4.1. Flowchart of the main program algorithm..... 38
Figure 4.2. Flowchart of the interrupt service routine (ISR) for processing encoder pulses. 39
Figure 4.3. Flowchart for calculating displacement and linear velocity..... 40
Figure 4.4. Flowchart of the algorithm for reading and processing tilt sensor data. 42
Figure 4.5. Flowchart of the PWM calculation algorithm for each wheel using the LQR controller. 43
Figure 4.6. Flowchart of the data acquisition and real-time plotting algorithm. . 45
Figure 4.7. Graphical User Interface (GUI) for system monitoring. 46
Figure 5.1. Hardware Connection Diagram. 47
Figure 5.2. Physical model of the Two-Wheeled Balancing Robot (TWBR). 48

Figure 5.3. Response of the pitch angle, tilt angle, and yaw angle in Case 1..... 49
Figure 5.4. Response of the pitch angle, tilt angle, and yaw angle in Case 2..... 49
Figure 5.5. Response of the pitch angle, tilt angle, and yaw angle in Case 3..... 50
Figure 5.6 The robot's equilibrium state on Gazebo 50
Figure 5.7. Physical prototype - Front view. 51
Figure 5.8. Physical prototype - Isometric/Perspective view. 51
Figure 5.9. Physical prototype - Side view in Low Posture..... 52
Figure 5.10. Physical prototype - Side view in Medium Posture. 52
Figure 5.11. Physical prototype - Side view in High Posture. 53
Figure 5.12 Shows the tilt angle and return position of the vehicle during operation. 54

LIST OF ABBREVIATIONS

Abbreviation	English Full Name
LQR	Linear Quadratic Regulator
PID	Proportional Integral Derivative
TWLR	Two-Wheeled Legged Robot
MIMO	Multiple Input Multiple Output
RAM	Ramdom Access Memory
ROM	Read Only Memory
EPROM	Erasable Programmable Read Only Memory
I/O	Input/Ouput
PWM	Pulse Width Modulation
UART	Universal Asynchronous Receiver - Transmitter
I2C	Inter – Intergrated Circuit
IMU	Inertial Measurement Unit
SRAM	Static Ramdom Access Memory
ICSP	In Circuit Serial Programming
SPI	Serial Peripheral Interface
DMP	Digital Motion Processor
ADC	Analog to Digital Converter
DOF	Degrees Of Freedom
DC	Direct Current

CHAPTER 1: INTRODUCTION

This chapter presents a general overview, the rationale for the project, and establishes the objectives and scope of the research. Consequently, it proposes the methodology and research direction for the development of the self-balancing two-wheeled robot.

1.1 Problem Statement

In the context of rapidly advancing automation, mobile robots are gradually transitioning from enclosed factory floors into human living environments. Applications range from sidewalk delivery robots and restaurant service robots to security patrol systems. However, human environments are inherently uneven. They consist of a complex mixture of smooth surfaces and numerous obstacles such as steps, door thresholds, steep inclines, and rough terrains. This presents a formidable challenge for mechanical designers: How can a robot maneuver rapidly and energy-efficiently on flat surfaces while possessing the capability to climb and overcome obstacles when necessary?

To address this locomotion problem, there are currently two predominant schools of robot design, each exhibiting completely contrasting advantages and disadvantages:

1.1.1 Advantages and Disadvantages of Wheeled and Legged Robots

First, regarding Wheeled Robots: This is the most fundamental and widely applied structure. Notably, among Mechatronics students, the Two-Wheeled Inverted Pendulum (TWIP) robot model is highly favored.

- Advantages: The wheeled mechanism enables exceptionally fast, smooth, and highly energy-efficient locomotion. The mechanical structure is relatively simple, easy to manufacture, and maintain. Furthermore, the zero-turning-radius capability (in-place rotation) of two-wheeled robots is highly advantageous in confined spaces.
- Disadvantages: The most critical weakness of wheeled systems is their near-zero terrain adaptability. Because the wheel axles are rigidly fixed, the robot's body cannot autonomously alter its center of gravity. Encountering a minor step or rough terrain can easily cause the wheels to slip, get stuck, or result in the robot losing balance and toppling over.

Second, regarding Legged Robots: To overcome the limitations of wheels, researchers have developed bio-inspired legged robots (such as quadruped robots or bipedal humanoids).

- Advantages: Exceptional terrain traversal capabilities. They can easily step over rubble, climb stairs, or navigate yielding surfaces where wheeled robots would be completely immobilized.
- Disadvantages: Conversely, legged robots possess highly complex mechanical structures and kinematics. When traversing flat surfaces, the continuous lifting and placing of legs severely limits their speed. Furthermore, they consume a massive amount of power merely to maintain joint torque for a stable standing posture, resulting in a very short operational runtime.

1.1.2 The Hybrid Solution: Variable-Leg-Length Wheel-Legged Robot

Recognizing the practical limitations mentioned above, a breakthrough solution has been proposed: combining the "speed of wheels" with the "agility of legs" into a single system. This is the fundamental principle behind the Wheel-Legged Robot.

Instead of relying solely on a rigid chassis and wheels, the robot is equipped with flexible, extendable leg joints (variable-leg-length mechanism). This combination yields a highly optimized locomotion paradigm:

- On flat terrain, the robot retracts its legs, lowers its center of gravity, and utilizes wheels for high-speed, energy-efficient travel.
- Upon encountering obstacles or steps, the leg joints autonomously extend, allowing the robot to jump over or effortlessly push the wheels onto higher elevations.
- Additionally, the leg joints act as an active suspension system, absorbing road vibrations to ensure that the main body-housing sensitive electronics and sensors—maintains an absolute state of balance.

Figure 1.1 Wheel-Legged Robot navigates various challenging terrains.

Figure 1.2. Wheel-Legged Robot for patrolling and security surveillance.

1.1.3 Control challenges: Why LQR-Fuzzy?

Despite having many advantages, this variable-leg-length wheel-legged robot makes it hard for us in the control programming stage. You just imagine, when the robot's legs retract or extend, all physical parameters like the center of gravity position and moment of inertia will change continuously. It is no longer a fixed constant like normal robots.

If we only use a traditional PID or LQR controller which is based on a "static" mathematical model, the robot will very easily vibrate or fall over every time it changes its leg height. Simply because the old control parameter set no longer matches the new body of the robot.

Because of that, I decided to aim for the combined LQR-Fuzzy control method. In which:

- LQR is still the core controller to keep optimal balance.
- Fuzzy Logic will play the role of a smart "supervisor". It will look at the actual height of the robot's legs to automatically "jump" parameters, updating the weight matrices for LQR immediately.

This combination helps the robot always stay in the most stable state, whether it is "standing tall" or "bending down" to overcome obstacles. This is also the reason I

chose the topic: "Design, fabrication and control of a variable-leg-length bipedal wheeled robot using a combined LQR-Fuzzy control approach" as my bachelor's graduation project.

1.2 Project Objectives

This topic I aim to complete a real robot model with the following specific objectives:

- About mechanics: Design and manufacture a leg mechanism capable of extending and retracting smoothly, ensuring rigidity when the robot moves and changes height suddenly.
- About algorithm: Successfully build an adaptive LQR-Fuzzy controller. The robot must automatically understand what height it is at to give suitable control feedback.
- About verification: Perform thorough simulation in a virtual environment before running on the real model to ensure safety for components and optimize parameters.

1.3 Project Limitations

Due to the limits of time and budget of a student project, I focus on solving the following core problems:

- Motion mechanism: The robot focuses on changing height in the vertical direction (raising/lowering the center of gravity) and keeping balance. Complex movements like jumping high or climbing stairs will only be studied preliminarily.
- Operating environment: Testing mainly in indoor environments, on flat surfaces or with low obstacles and moderate slopes.
- Autonomous features: The project focuses on the dynamic balancing problem, not yet going deep into image processing or autonomous moving mapping (SLAM) problems.

1.4 Research Methodology

Based on practical knowledge, the mathematical model of the balancing robot has been clearly specified. The team proceeds to use a suitable controller to help the vehicle keep balance according to the set quality requirements.

Build the theoretical model of the self-balancing robot, simulate on MATLAB Simulink and perform calculations of parameters for the LQR controller.

Build the real model including:

- Mechanical design: the frame of the model.
- Application of electronic circuits, power circuits (wheel control).
- Sensor circuits (angle, velocity, acceleration) and sensor value compensation methods to estimate and filter noise.
- Central controller.
- Microprocessor programming.

1.5 Project Contents

The rest of the project has the following contents:

Chapter 2: Theoretical Background

Chapter 3: Design and Simulation

Chapter 4: Real Model Design

Chapter 5: Experimental Results and Conclusion

CHAPTER 2: THEORETICAL BACKGROUND

In this chapter, perform modeling and give the dynamic characteristics of the self-balancing two-wheeled vehicle system. State some controllers that can be used for the system, consider the pros and cons of each controller to choose a suitable controller. Through that, survey the Linear Quadratic Regulator (LQR) controller and build the system based on that controller.

2.1 Dynamic characteristics

Build the state equation system describing the two-legged wheeled robot system.

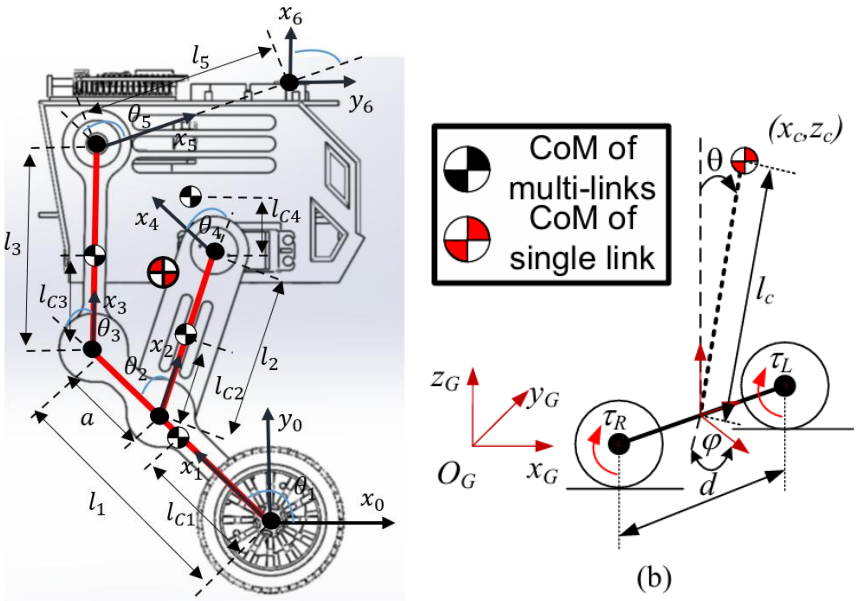


Figure 2.1. Diagram of different types of robot dynamics models.

Symbol – [Unit]	**Description**
$\theta_w(rad)$	Rotation angle of the wheel
$\theta_b(rad)$	Tilt angle of the body of the equivalent WIP model
$r - [m]$	Radius of the driving wheel
I_b	Total moments of inertia of above the driving wheel in the TLWR model
l_c	Distance between the center of mass (CoM) position and wheel axis in the TLWR m
d	Distance between two wheels
m_1	Mass of the shank

m_2	Mass of the lower thigh
m_3	Mass of the upper thigh
m_4	Mass of the body
l_1	Length of the shank
l_2	Length of the lower thigh
l_3	Length of the upper thigh
l_4	Distance between the coordinate system 4th and 5th
l_5	Distance between the coordinate system 5th and 6th
τ_L	Torque of the left wheel
τ_R	Torque of the right wheel
lc_1	Position of CoM of the shank
lc_2	Position of CoM of the lower thigh
lc_3	Position of CoM of the upper thigh
lc_4	Position of CoM of the body
θ_1	The joint angle between the coordinate system 0th and 1th
θ_2	The joint angle between the coordinate system 1th and 2th
θ_3	The joint angle between the coordinate system 2th and 3th
θ_4	The joint angle between the coordinate system 3th and 4th
θ_5	The joint angle between the coordinate system 4th and 5th
θ_6	The joint angle between the coordinate system 5th and 6th
a	Distance between the coordinate system 2th and 3th

Table 2.1. Symbols and descriptions of the system variables.

2.2 Equivalent center of gravity calculation

The position of the equivalent center of gravity is weighted by the mass of each joint and their center of gravity position. To establish the relationship between this center of gravity and the axis coordinate system, the Denavit-Hartenberg (D-H) convention was used to establish the kinematic model. By establishing the coordinates, the D-H parameters of the TLWR are shown in Table 2.2.

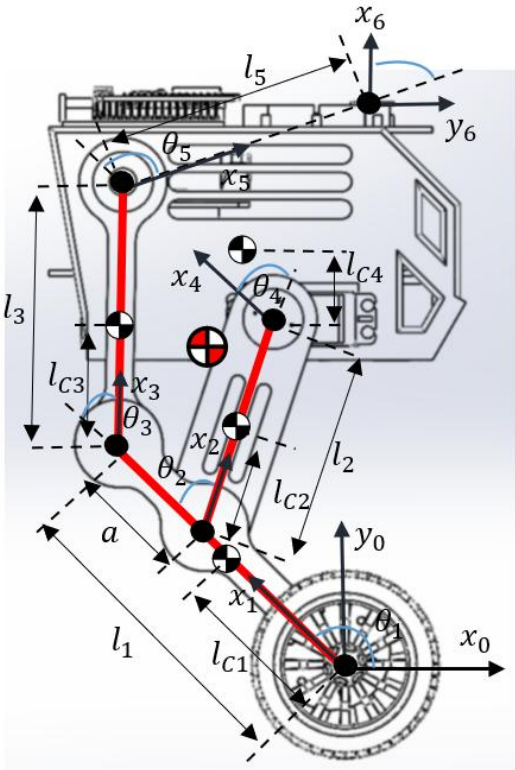


Figure 2.2. Kinematic parameters of the TLWR (Two-Wheeled Legged Robot).

Table 2.2 DH-PARAMETERS OF THE TLWR

Coordinate	a_{i-1}	α_{i-1}	d_i	θ_i
1	0	0	0	θ_1
2	$l_1 - a$	0	0	θ_2
3	l_2	0	0	θ_4
4	l_4	0	0	θ_5
5	l_5	0	0	θ_6

The homogeneous transformation matrix from the i-th coordinate to the (i-1)-th coordinate is given as (1)

$$ {}^{i-1}_{i}T = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i \cos\alpha_{i-1} & \cos\theta_i \cos\alpha_{i-1} & -\sin\alpha_{i-1} & -d_i \sin\alpha_{i-1} \\ \sin\theta_i \sin\alpha_{i-1} & \cos\theta_i \sin\alpha_{i-1} & \cos\alpha_{i-1} & d_i \cos\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.1} $$

The homogeneous transformation matrix between the i-th coordinate system and the wheel axis coordinate system is:

$$ {}^{w}_{i}T = {}^{w}_{1}T\,{}^{1}_{2}T \ldots {}^{i-1}_{i}T \tag{2.2} $$

The position of the CoM of the upper body relative to the axis coordinate system can be presented as follows:

$$ {}^{w}P_{C}(q_b) = \frac{\sum_{i=1}^{n} m_i\, {}^{w}P_{Ci}(q_b)}{\sum_{i=1}^{n} m_i} \tag{2.3} $$

$$ {}^{w}P_{Ci}(q_b) = {}^{w}_{i}T(q_b)\ {}^{i}P_{Ci} \tag{2.4} $$

Where: m_i: is the mass of the i-th joint.

$q_b = \left[q_{b_1}, \ldots, q_{b_4}\right]^T$: are the actual angles.

Based on the coordinates of the CoM, the pendulum length lc and the tilt angle θ of the inverted pendulum can be determined as in equations (2.5) and (2.6).

$$ l_c = \sqrt{x_c^2 + z_c^2} \tag{2.5} $$

$$ \theta = a\tan\left(\frac{x_c}{z_c}\right) \tag{2.6} $$

2.3 The inverted pendulum problem

2.3.1 Parameters of the inverted pendulum system

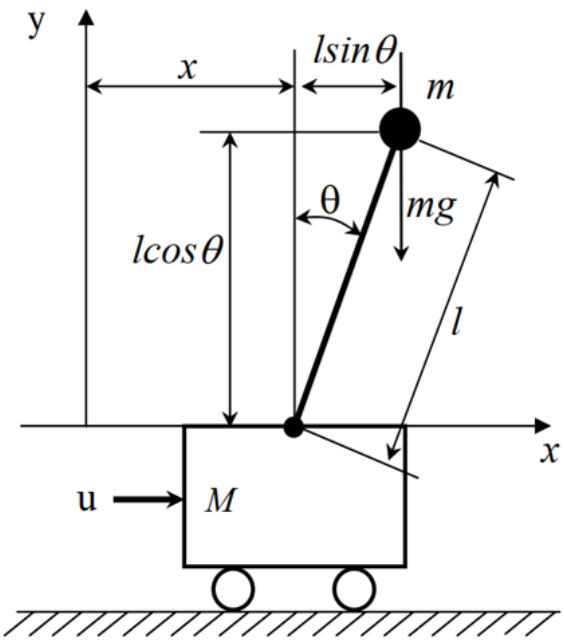


Figure 2.3. Mathematical model of the inverted pendulum on a vehicle system.

Parameters of the inverted pendulum system

M : Mass of the vehicle

m : Mass of the pendulum

l : Length of the pendulum

u : Force applied to the vehicle$[N]$

g: Gravity acceleration $[m/s^2]$

x : Position of the vehicle$[m]$

θ : Angle between the pendulum and the vertical direction $[rad]$

2.3.2 Mathematical model of the inverted pendulum system

Using the Euler - Lagrange formula:

Kinetic energy of the heavy mass at the end of the pendulum:

$$T_P = \frac{1}{2}m\dot{x}_P^2 + \frac{1}{2}m\dot{y}_P^2 = \frac{1}{2}m\left(\dot{x} + l\dot{\theta}\cos\theta\right)^2 + \frac{1}{2}m\left(-l\dot{\theta}\sin\theta\right)^2$$

$$\Rightarrow T_P = \frac{1}{2}m\dot{x}_P^2 + ml\dot{x}\dot{\theta}\cos\theta + \frac{1}{2}ml^2\dot{\theta}^2 \tag{2.7}$$

Kinetic energy of the vehicle:

$$T_c = \frac{1}{2}M\dot{x}^2 \tag{2.8}$$

Kinetic energy of the system:

$$T = T_P + T_C = \frac{1}{2}(M+m)\dot{x}^2 + ml\dot{x}\dot{\theta}\cos\theta + \frac{1}{2}ml^2\dot{\theta}^2 \tag{2.9}$$

Potential energy of the system:

$$U = mgl\cos\theta \tag{2.10}$$

Lagrangian operator:

$$L = T - U = \frac{1}{2}(M+m)\dot{x}^2 + ml\dot{x}\dot{\theta}\cos\theta + \frac{1}{2}ml^2\dot{\theta}^2 - mgl\cos\theta \tag{2.11}$$

Euler-Lagrange equation:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right) - \frac{\partial L}{\partial x} = F \tag{2.12}$$

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) - \frac{\partial L}{\partial \theta} = 0 \tag{2.13}$$

Substitute (2.11) into (2.12) and (2.13):

$$(M+m)\ddot{x} + ml\cos\theta\ddot{\theta} - ml(\sin\theta)\dot{\theta}^2 = u \tag{2.14}$$

$$m\ddot{x}\cos\theta + ml\ddot{\theta} - mg\sin\theta = 0 \tag{2.15}$$

Therefore

$$\ddot{x} = \frac{u + ml(\sin\theta)\dot{\theta}^2 - mg\cos\theta\sin\theta}{M + m - m(\cos\theta)^2} \tag{2.16}$$

$$\ddot{\theta} = \frac{u\cos\theta - (M+m)g\sin\theta + ml(\cos\theta\sin\theta)\dot{\theta}^2}{ml(\cos\theta)^2 - (M+m)l} \tag{2.17}$$

2.3.3 Establish the system of equations.

Proceed to convert the equation found in (1.16) to the standard form:

$$\frac{d}{dt}x = f(x, y, t) = \frac{d}{dt}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \frac{d}{dt}\begin{bmatrix} \theta \\ \dot{\theta} \\ x \\ \dot{x} \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} \tag{2.18}$$

Set the state variables:

$$x_1 = \theta, x_2 = \dot{\theta}$$

$$x_3 = x, x_4 = \dot{x}$$

Where:

$$f_1 = x_2$$

$$f_2 = \frac{u\cos x_1 - (M+m)g\sin x_1 + ml(\cos x_1 \sin x_1)x_2^2}{ml(\cos x_1)^2 - (M+m)l} \tag{2.19}$$

$$f_3 = x_4$$

$$f_4 = \frac{u + ml(\sin x_1)x_2^2 - mg\cos x_1 \sin x_1}{M + m - m(\cos x_1)^2} \tag{2.20}$$

Nonlinear state equation:

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \\ \dot{x_3} \\ \dot{x_4} \end{bmatrix} = \begin{bmatrix} x_2 \\ \dfrac{u\cos x_1 - (M+m)g\sin x_1 + ml(\cos x_1 \sin x_1)x_2^2}{ml(\cos x_1)^2 - (M+m)l} \\ x_4 \\ \dfrac{u + ml(\sin x_1)x_2^2 - mg\cos x_1 \sin x_1}{M + m - m(\cos x_1)^2} \end{bmatrix} \tag{2.21}$$

Linearized equation around the equilibrium point ($x_0 = 0, u_0 = 0$)

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \\ \dot{x_3} \\ \dot{x_4} \end{bmatrix} = J_x(x_0, u_0)\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + J_u(x_0, u_0)u \tag{2.22}$$

With:

$$J_x(x_0,u_0)=\begin{pmatrix}\frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \frac{\partial f_1}{\partial x_4}\\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \frac{\partial f_2}{\partial x_4}\\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} & \frac{\partial f_3}{\partial x_4}\\ \frac{\partial f_4}{\partial x_1} & \frac{\partial f_4}{\partial x_2} & \frac{\partial f_4}{\partial x_3} & \frac{\partial f_4}{\partial x_4}\end{pmatrix}=\begin{pmatrix}0 & 1 & 0 & 0\\ \frac{(M+m)g}{Ml} & 0 & 0 & 0\\ 0 & 0 & 0 & 1\\ \frac{-mg}{M} & 0 & 0 & 0\end{pmatrix} \tag{2.23}$$

And:

$$J_u(x_0,u_0)=\begin{bmatrix}\frac{\partial f_1}{\partial u}\\ \frac{\partial f_2}{\partial u}\\ \frac{\partial f_3}{\partial u}\frac{\partial f_4}{\partial u}\end{bmatrix}=\begin{bmatrix}0\\ \frac{-1}{Ml}\\ 0\\ \frac{1}{M}\end{bmatrix} \tag{2.24}$$

Thus, substituting (2.22) and (2.23) into (2.24) and the parameters of the problem, we establish the linearized equation around the equilibrium point ($x_0=0, u_0=0$)

$$\begin{bmatrix}\dot{x_1}\\ \dot{x_2}\\ \dot{x_3}\\ \dot{x_4}\end{bmatrix}=\begin{bmatrix}0 & 1 & 0 & 0\\ \frac{(M+m)g}{Ml} & 0 & 0 & 0\\ 0 & 0 & 0 & 1\\ \frac{-mg}{M} & 0 & 0 & 0\end{bmatrix}\begin{bmatrix}x_1\\ x_2\\ x_3\\ x_4\end{bmatrix}+\begin{bmatrix}0\\ \frac{-1}{Ml}\\ 0\\ \frac{1}{M}\end{bmatrix}u \tag{2.25}$$

2.4 Some control methods that can be used for the system

2.4.1 Proportional Integral Derivative controller (PID controller)

The PID controller is a closed-loop controller, having a combination of three controllers: proportional, integral, and derivative, capable of adjusting the error as low as possible, increasing response speed, reducing overshoot, and limiting oscillation. The controller is easy to design with us only needing to tune 3 parameters of the PID controller which are K_p, K_i, K_d.

Transfer function of the PID controller:

$$G_{PID}(s)=K_P+\frac{K_I}{s}+K_D s=K_P(1+\frac{1}{T_1 s}+T_D s) \tag{2.26}$$

In terms of advantages:

- Easy to design because we only adjust three parameters K_p, K_i, K_d

- .In terms of disadvantages:

- The fact that we design the PID controller we have to find three parameters of PID, with a MIMO system we need many PID controllers to control, with the fact that we use many PID controllers we need to find many parameters. In reality, the 3 parameters Kp, Ki, Kd are hard for us to find exactly but must rely on experience.
- The PID controller when we change the basic parameters of the system, the 3 parameters we found are not accurate and the stability of the system is changed accordingly.

2.4.2 Proportional Integral Derivative controller combined with Genetic Algorithm

With the PID controller introduced earlier, it has many advantages like finding the parameters of the controller we must rely on experience or when we change the mass of the object it makes the stability of the system no longer stable. The Genetic Algorithm (GA) algorithm is a random optimization algorithm based on the mechanism of natural selection and genetic evolution. Applied first in 2 main fields: optimization and machine learning. It is applied to the PID controller to improve the fact that when we change the values of the system, GA will find and give the optimal value for the system.

In terms of advantages: Helps the system can operate when we change the values of the system such as: mass, size,...

In terms of disadvantages: The time to calculate the data relies heavily on hardware, the results found can make the RAM of the microcontroller overflow and cannot be calculated.

2.4.3 Fuzzy controller

Fuzzy control is a control method that imitates the process of processing unclear information and making control decisions of humans. The mathematical basis of the fuzzy controller was introduced by Zadeh since 1965. Today, it is used more

widely in the field of automatic control. Fuzzy inferences are built thanks to the logical reasoning of humans. A basic fuzzy controller has a structure as follows:

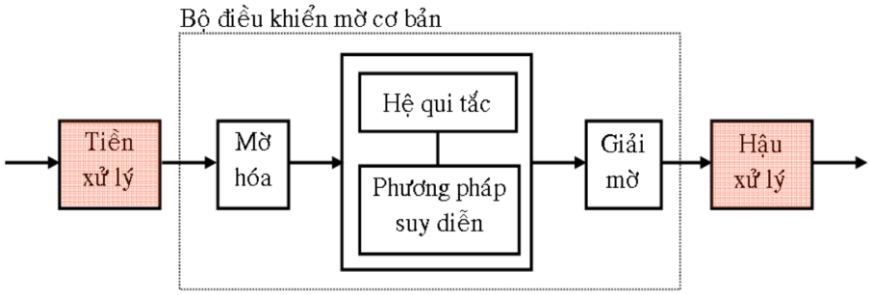


Figure 2.4. Basic structure of a Fuzzy Logic Controller.

- Pre-processing: normalize or scale the measured value into the standard value range, filter noise.
- Fuzzification: convert the crisp value fed back from the object's output into a fuzzy value so the rule system can infer.
- Fuzzy rule system: drawn from expert experience in controlling the object.
- Inference method: from the built rule system there will be methods to find the corresponding output through the defuzzification block.
- Defuzzification: convert the inferred fuzzy value at the output of the rule system into crisp values to control the object.
- Post-processing: scale the value after defuzzification into a physical value to control the system.

In terms of advantages:

- Easy to design because the fuzzy set and fuzzy propositions are built based on human reasoning.
- High flexibility.
- Because it was born a long time ago, fuzzy rules can be easily found on the network and changed to suit the real system.
- Can link with the PID controller to increase the stability of the system.

Disadvantages:

- If our reasoning is wrong, the output result can be wrong.
- The defuzzification process depending on which model we choose can make the accuracy of the given result incorrect.

- With a complex multi-input multi-output system, we need many different base sets and different fuzzy rules. This can make the system more troublesome to design.
- Due to reasoning to design, the nature and model of the system are not clearly grasped.
- Requires a microcontroller with fast, good processing speed and high memory.

2.4.4 Optimal controller

A control system designed in the best working mode is a system always in an optimal state according to a certain quality standard (achieving extreme value). There are many optimal control problems, depending on:

- Type of control object.
- Continuous or discrete time domain.
- Quality criteria.
- Optimal problem has constraints or not.

There are two main optimal control methods:

- Static optimal control: quality criteria do not depend on time.
- Dynamic optimal control: quality criteria depend on time, having the following methods:

- Linear Quadratic Regulator (LQR) problem.
- H2 optimal control problem.
- ...

Survey the advantages and disadvantages of the optimal controller using the Linear Quadratic Regulator (LQR) method as follows:

In terms of advantages:

- The system is still stable even when we change the characteristics on the system.
- Can be designed with complex nonlinear systems.
- The given result has high stability.

In terms of disadvantages:

- Matrix calculation of the LQR controller is very complex.
- Choosing the appropriate weight matrix in the quality criteria is very important because it greatly affects the calculated result.

- If affected by system noise and measurement noise acting on the system, it will cause difficulty when balancing, the control signal is unstable with many spikes causing effects on the motor.

- Through analyzing the advantages and disadvantages of each controller presented above, the team decided to design and simulate the self-balancing bipedal robot system using a Combined LQR-Fuzzy Control approach. This hybrid controller leverages the optimal state-space regulation of LQR while utilizing Fuzzy Logic to adaptively adjust control parameters, ensuring stability across various leg-length configurations and dynamic postures.

2.5 LQR controller design

2.5.1 Modeling the inverted pendulum system by mathematical equations

Based on the mathematical equation built in the previous section, we proceed to build the inverted pendulum system model as follows:

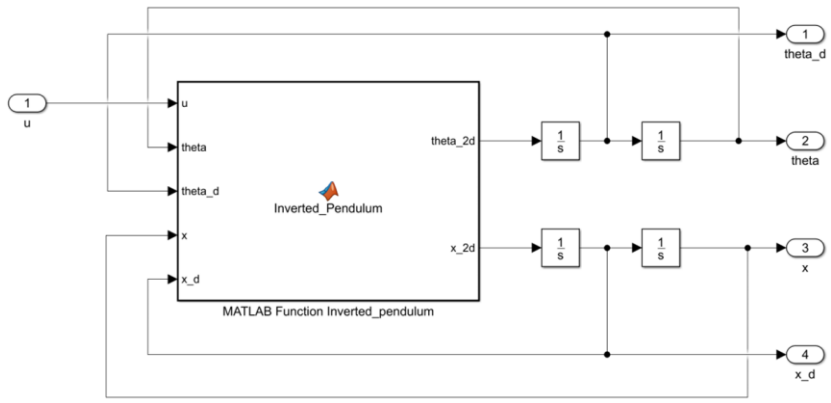


Figure 2.5. Modeling of the inverted pendulum on a cart system in Simulink.

2.5.2 Theoretical Background

The dynamic characteristics of the inverted pendulum system can be described by the linear state variable equation system. This is only true when the deviation angle is small. Full state feedback system, meaning 4 state variables can be measured (deviation angle, angular velocity, vehicle position, vehicle velocity). And there is no noise acting on the system.

2.5.3 Design of the LQR controller

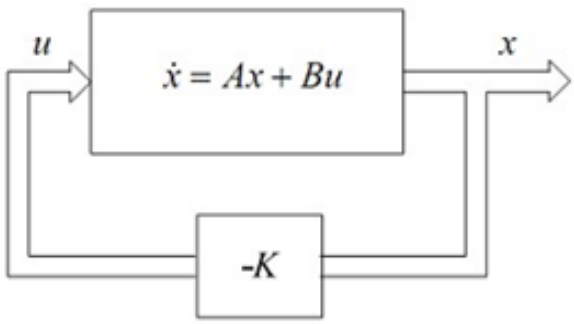


Figure 2.6. Block diagram of the Linear Quadratic Regulator (LQR) controller.

In this topic, only consider the infinite-time continuous LQR problem.

The linear object is described by the state equation:

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{2.27}$$

Corresponding to

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \\ \dot{x_3} \\ \dot{x_4} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{(M+m)g}{Ml} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ \frac{-mg}{M} & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{-1}{Ml} \\ 0 \\ \frac{1}{M} \end{bmatrix} u$$

$$\Rightarrow A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{(M+m)g}{Ml} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ \frac{-mg}{M} & 0 & 0 & 0 \end{bmatrix} \tag{2.28}$$

$$B = \begin{bmatrix} 0 \\ \frac{-1}{Ml} \\ 0 \\ \frac{1}{M} \end{bmatrix} \tag{2.29}$$

Quadratic quality criteria, in which the final time approaches infinity:

$$J(u) = \frac{1}{2}\int_0^{\infty} [x^T(t)Qx(t) + u^T(t)Ru(t)]dt \tag{2.30}$$

At this time we have the weight matrices as follows:

$$Q = \begin{bmatrix} Q_1 & 0 & 0 & 0 \\ 0 & Q_2 & 0 & 0 \\ 0 & 0 & Q_3 & 0 \\ 0 & 0 & 0 & Q_4 \end{bmatrix} \tag{2.31}$$

$R = R_1$

With parameters Q_1, Q_2, Q_3, Q_4, R_1 to fine-tune the controller. Where parameters Q_1, Q_2, Q_3, Q_4, are the optimal weights for the deviation angle Θ, angular velocity $\dot{\theta}$, vehicle position x , vehicle velocity $\dot{x}$ and parameter R_1 is the optimal weight for the control signal for the left wheel v_l and the control signal for the right wheel v_r. If we want a state variable to be prioritized in the control, we set the corresponding weight parameter to be relatively larger than the other weights in the two matrices Q and R.

After selecting the corresponding control parameters, we proceed to build the feedback parameter K with the control signal $u = -Kx(t)$. The parameter K is calculated based on algebraic equation **Error! Reference source not found.**.

We will proceed to select the parameters Q and R as follows

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.32}$$

$$R = 1$$

Optimal control signal:

$$u^*(t) = -Kx(t) \tag{2.33}$$

Where:

$$K = R^{-1}B^T P \tag{2.34}$$

And P is the positive semi-definite solution of the algebraic Riccati equation:

$$PA + A^T P + Q - PBR^{-1}B^T P = 0 \tag{2.35}$$

Note: in this case K and P do not depend on time.

Minimum value of the quality criteria:

$$J_{\min} = x^T(0)Px(0) \tag{2.36}$$

Solution of the continuous linear quadratic regulator:

$$K = lqr(A, B, Q, R) \tag{2.37}$$

CHAPTER 3: DESIGN AND SIMULATION

In this chapter, we will give the requirements to design the hardware for the system. We will give suitable devices for the system then from the advantages and disadvantages of each device and compared to the set requirements to choose the most suitable device considering both economic and technical aspects. After having chosen the hardware devices, we will give the total wiring diagram board of the system.

3.1 Hardware design

3.1.1 Design requirements

To design the hardware ensuring for the vehicle to operate, we need to choose devices suitable with the following requirements:

- Ensure the total mass of the vehicle so the motor can pull to balance.
- Power supply is enough for the system to operate.
- The vehicle frame is sturdy.
- The sensor returns signals to avoid maximum noise and error.
- Survey operation in a short time.
- Motor torque is enough to pull the load.
- Microcontroller has full communication standards with the computer and sensors, ensuring fast signal processing.

3.1.2 Hardware device selection

a. Central control block

The central control block is a microcontroller having the task of processing data read from the tilt angle sensor (IMU) and rotary encoder (Encoder), from there performing complex control algorithms to output signals to the Motor control block.

The general structure of the microcontroller includes: Microprocessor is connected to peripheral ports, I/O pins, Analog pins, serial communication port, interrupt functions (Interrupt) to read encoder signals, timer (Timer/Counter), I2C protocol to transmit/receive data from the sensor, power supply pins and storage memories like Flash, SRAM, EEPROM.

The central control block must have fast processing speed with high working efficiency to meet the real-time of the balancing system. To implement this topic,

we use the ESP32 DevKit V1 (NodeMCU-32S) board with the following outstanding advantages:

- Extremely high processing speed: Uses a dual-core microprocessor with a clock speed of up to 240MHz, helping to perform matrix math in the LQR algorithm and Kalman filter many times faster than 8-bit microcontroller lines.
- Comprehensive peripheral interrupt support: All I/O pins of ESP32 have the ability to be configured as interrupt pins, allowing reading signals from the Encoder flexibly to control the speed and position of the motor accurately.
- Integrated wireless connection: Has available Wi-Fi and Bluetooth Dual-mode, strongly supporting in transmitting real data to the computer or phone to monitor (Monitor) and adjust the K parameters of the controller remotely without plugging in wires.
- High PWM resolution: ESP32 supports PWM pulse generator (LEDC) with configurable resolution (up to 16-bit), helping to control the motor smoother and quieter.
- Large memory: Abundant Flash and RAM capacity allows storing complex programs and processing large data.
- Reasonable price and large community: Is the most popular microcontroller line today for IoT and Robotics applications with rich support libraries on the Arduino IDE platform.
- Modern operating voltage: Operates at 3.3V logic level, suitable for most modern IMU sensors, helping to minimize noise and save energy.
- Technical specifications:
- Main microcontroller: ESP32-WROOM-32 (Dual-core Xtensa® 32-bit LX6).
- Operating frequency: 240 MHz.
- Power supply: 5VDC from Micro-USB port or 3.3V - 12VDC through VIN pin.
- Number of Digital I/O pins: 34 pins (most can be used as PWM, I2C, SPI, UART).
- Number of Analog Input pins (ADC): 18 pins (12-bit resolution).
- Flash Memory: 4 MB.
- SRAM: 520 KB.
- Number of hardware UART ports: 3 ports.

- Supported protocols: Wi-Fi 802.11 b/g/n, Bluetooth v4.2 BR/EDR and BLE, I2C, SPI, UART, CAN bus.

- Dimensions: About 52 x 28 mm (compact, saving space for the vehicle frame).

- Mass: About 10g.

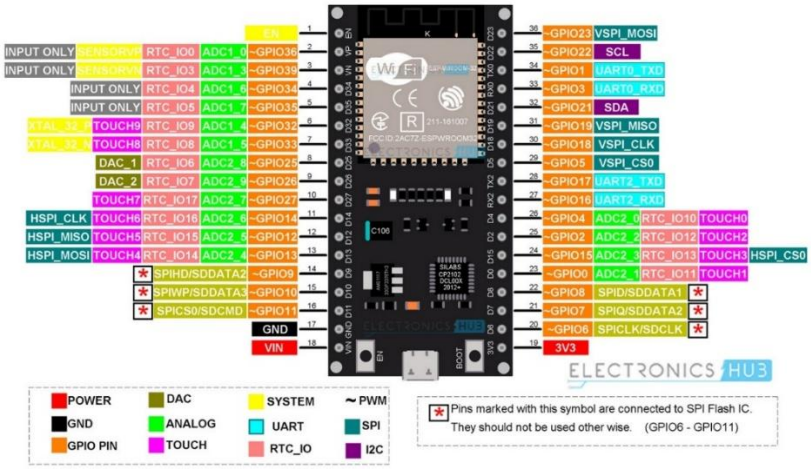


Figure 3.1. Pinout diagram of the ESP32 microcontroller.

b. Tilt angle sensor block

The tilt angle sensor block is a module consisting of two sensors which are:

- Accelerometer: sensor measuring the acceleration of the module (including gravity acceleration), having three xyz axes corresponding to three spatial dimensions.

- Gyroscope: sensor measuring the rotation speed of the module around an axis, having three xyz axes corresponding to three spatial dimensions.

The central control block will receive the return signal from these two sensors and will process and calculate the tilt angle value of the vehicle.

For the system requirements, only to determine the tilt angle value of the vehicle, we chose the GY-521 6DOF IMU MPU6050 sensor type with the following advantages and disadvantages:

In terms of advantages:

- It is the most popular sensor type on the market with a reasonable price.

- Has built-in hardware specialized in signal processing (Digital Motion Processor – DMP) collected by the sensor and performs necessary calculations helping to improve processing speed, faster response and significantly reduce the calculation processing part of the microcontroller.

- I2C communication is suitable for the chosen Central control block.

- Has many library sources to support in signal processing and programming.

In terms of disadvantages:

- The return signal from the two sensors has errors on each axis making the measured values deviate compared to reality.

- Both the acceleration and rotation speed measuring sensors are affected by noise.

=> To be able to overcome this disadvantage, we use the Kalman filter to process algorithms to estimate the return state from the two sensors most accurately. The Kalman filter we use has been pre-written in the form of a library for the microcontroller Technical specifications:

- Includes 6 sensor axes (6DOF): acceleration sensor (3 axes), gyroscope sensor (3 axes).

- Analog to Digital Converter (ADC): 16-bit.

- Operating voltage: 3-5 VDC.

- Communication voltage: 3-5 VDC.

- Communication standard: I2C.

- Gyroscope value in the range: ±250, ±500, ±1000, ±2000 °/sec.

- Accelerometer value in the range: ±2, ±4, ±8, ±16 g.

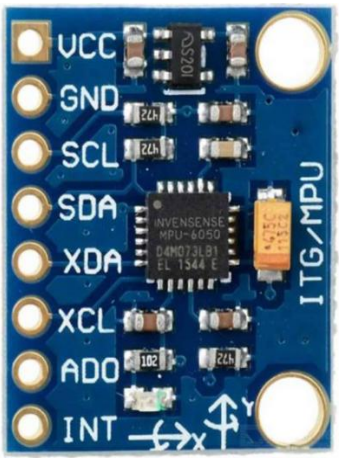


Figure 3.2. MPU6050 IMU tilt sensor.

Pin	**Connection and Function**
VCC	Positive power supply pin.
GND	Ground (GND) pin.
SCL	Serial Clock Line
SDA	Serial Data Line
XDA	Data pin (connect with other sensors)

XCL	Pulse pin (connect with other sensors)
ADO	Bit0 of I2C address
INT	Interrupt pin

Table 3.1. Pin connection table for the MPU6050 sensor

c. Motor control block

The motor control block receives PWM (Pulse Width Modulation) signals from the Central control block to easily control the two motors for the left and right wheels to rotate forward, reverse, stop, or follow fast or slow speeds so the vehicle can easily reach a balanced state.

Pulse Width Modulation (PWM) is a modulation technique used to encode a message into a pulse signal. Although this modulation technique can be used to encode information for transmission, its main use is to allow controlling the power supplied to electrical devices, especially for inertial loads like motors.

The average value of voltage (and current) supplied to the load is controlled by changing the switching between the source and the load off at a very fast speed. The longer the closing time compared to the cutting time, the higher the total power supplied to the load.

Here, we choose the BTS7960 DC motor control circuit (Dual Full Bridge Driver) with the following advantages:

Extremely high power: Continuous load current capacity up to 43A, responding well to 775 DC motors or high-power reduction motors commonly used in medium-sized balancing vehicles.

- Integrated isolation circuit: The module has a 74HC244 buffer IC to isolate the control signal from the microcontroller (such as ESP32) from the power section, protecting the microcontroller from noise pulses or motor incidents.
- Comprehensive protection mechanism: Integrated features for over-temperature, over-voltage, under-voltage, and short-circuit protection.
- High response speed: Supports PWM frequency up to 25kHz, helping the motor operate smoothly and reducing mechanical noise.

Disadvantages:

- Large size: Occupies more space compared to low-power drivers like L298N.

- Number of control pins: Needs more signal pins (usually needs 4 pins: 2 PWM pins and 2 Enable pins) to control one motor.

Technical specifications:

- Motor supply voltage (Vcc): 6V – 27V DC
- Control logic voltage (Vct): 3.3V – 5V DC
- Maximum load current (Peak): 43A
- Continuous load current: ~20A - 30A (depending on heat dissipation conditions)
- Maximum PWM frequency: 25 kHz
- Internal resistance ($R_{DS(on)}$): 16 mΩ (very low, helping to reduce heat dissipation)
- Standby current consumption: < 7 µA
- Functional pins: RPWM, LPWM, R_EN, L_EN, VCC, GND
- Dimensions: 50 x 50 x 43 mm
- Weight: 66g

Figure 3.3. BTS7960 DC Motor Driver Module – 43A High-power Motor Driver

Pin	Connection and Function
VCC	Logic power supply for the buffer IC (74HC244). Connect to the **3.3V** pin of the ESP32.
GND	Common ground pin (Ground). Connect to the **GND** pin of the ESP32.
R-EN	(Right Enable) Allows the right H-bridge half to operate. Connect to high level (3.3V) or a Digital pin to turn on/off.
L-EN	(Left Enable) Allows the left H-bridge half to operate. Connect to high level (3.3V) or a Digital pin to turn on/off.

RPWM	(Right PWM) Controls the speed and rotation direction on the right. Connect to the **PWM** pin of the ESP32.
LPWM	(Left PWM) Controls the speed and rotation direction on the left. Connect to the **PWM** pin of the ESP32.
R-IS	(Right Isense) Over-current/over-temperature warning on the right side. Can be left empty (or connect to an Analog pin for monitoring).
L-IS	(Left Isense) Over-current/over-temperature warning on the left side. Can be left empty (or connect to an Analog pin for monitoring).

Table 3.2. Pin connection table for the BTS7960 DC Motor Driver.

d. Motor block and motor pulse reading sensor

i. Wheel motor block and motor pulse reading sensor includes:

- Two motors on the right and left are the main actuators of the vehicle to control the vehicle's balance.

- Two rotary encoders attached to each motor help feed pulses back to the Central control block, from there processing signals to help accurately determine the position, rotation speed, and rotation direction of each motor.

Because the requirement is a motor with a built-in rotary encoder and a motor that pulls a load of about 2kg, we have chosen the JGB37-520 12V 333RPM Gear Motor with encoder with the following advantages:

- Integrated Encoder: Makes measuring speed and travel distance easy without installing additional separate sensors, which is very important for PID and LQR algorithms in balancing vehicles.

- Metal gearbox: All gears in the reduction gearbox are made of steel, helping to increase durability, bear loads well, and minimize gear wear after long-term use.

- High stability: The motor operates smoothly, with large torque at low speed ranges, helping the robot keep balance better when there are changes in load.

- The motor uses high-quality materials: with pure copper wire cores, 407 steel sheets, strong magnetic magnets, etc., providing superior power and durability compared to other cheap types on the market.

Technical specifications:

- Motor:

- Model: JGB37-520.
- No-load speed (at 12V): 333 revolutions per minute (RPM).

- No-load current: ~0.15A.
- Rated torque: ~4.5 kg.cm.
- Power: 10W.
- Gearbox reduction ratio: 1:30 (Common ratio for 333RPM level).
- Shaft diameter: 6mm (D-shaped shaft helps avoid wheel slippage).
- Shaft length: 15mm.
- Body size: 37mm diameter, length depends on the version.

- Encoder:

- Power supply voltage for Encoder operation: 3.3 – 5 VDC.
- Hall effect magnetic sensor, having 2 phase-shifted AB channels.
- Encoder disk returns 11 pulses / 1 channel / 1 revolution.
- Number of Encoder pulses per channel per 1 main shaft revolution: 11 x 34 = 374 pulses.

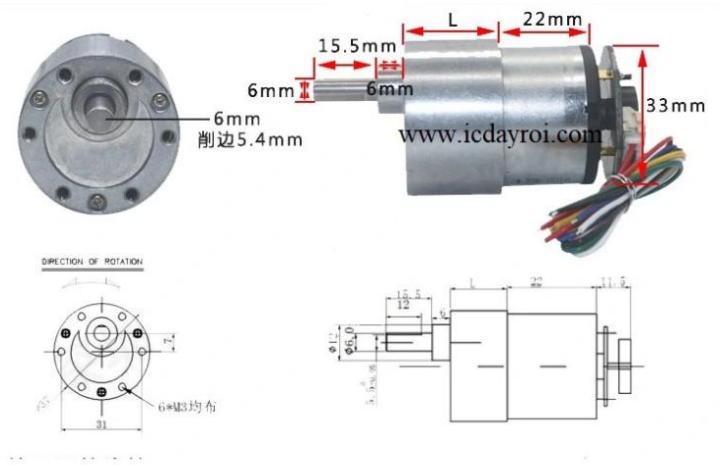


Figure 3.4. GM25-370 DC Geared Motor with Encoder.

Pin	Connection and Function
M1 (red)	Positive power supply wire for the motor
GND (black)	Negative power supply wire for the motor
C1/A (yellow)	Channel A pulse feedback signal
C2/B (green)	Channel B pulse feedback signal
VCC (blue)	Power supply wire for the Encoder
M2 (white)	Negative power supply wire for the motor

Table 3.3. Pin connection table for the DC motors and Encoders.

ii. Hip motor block

- To allow the robot to move flexibly through many terrains and change height smoothly, on the two hips of the robot body, we use two Digital RC Servo TD-8120MG 270-degree motors which have the following advantages:

- Extremely large Torque: With a torque up to 20kg.cm, this Servo easily controls heavy mechanisms without encountering weakness.

- High accuracy: As a Digital Servo line, it has the ability to respond to PWM signals quickly, with small angle error and extremely stable position holding.

- Expanded 270-degree rotation angle: Provides a more flexible operating range compared to traditional 180-degree lines, suitable for robot joints or steering systems.

- Good heat dissipation capability: The outer shell has a middle part made of aluminum, helping fast heat dissipation for the internal motor when working continuously at high intensity.

- Technical specifications:

- Model: TD-8120MG (270° version)

- Operating voltage: 4.8V – 6.6V DC

- Stall Torque: 18.5 kg.cm (4.8V) / 21.8 kg.cm (6.6V)

- Operating speed: 0.18 sec/60° (4.8V) / 0.14 sec/60° (6.6V)

- Gear type: Metal Gears

- Maximum rotation angle: 270 degrees

- Shell material: ABS plastic combined with aluminum heat sink

- Weight: About 65g

- Dimensions: 40.0 x 20.5 x 40.5 mm

Figure 3.5. TD-8120MG 270-degree Digital RC Servo Motor.

3.1.3 Power supply block

Providing stable power for the blocks helps the system operate well. Can supply power to the system in many ways such as batteries, switching power supplies, or AC-DC adapters, etc.

Selection of the power supply block includes:

- Power source: consists of a 3S 5400mAh 30C Lipo battery.
- Buck converter: DC - DC LM2596 3A used for ESP32.
- Rocker switch: 10x15 mm.
- Buck DC-DC converter module: XL4015 for the servos.

Advantages of the chosen power supply block:

- When using a battery combined with a buck converter, it will help stabilize a fixed voltage level supplied to the system, in case of voltage drop when the battery is low.
- Can be used flexibly, more compact than using a switching power supply or adapter.
- Easy installation and wiring, not entangled by power cords when the vehicle runs.
- Easy to use, replace, and charge the battery for the power source.
- Using a switch helps be more convenient in turning the system on/off.
- Partly helps increase the mass of the vehicle, making the vehicle easier to balance.
- The LM2596 buck circuit has a compact size, capable of reducing voltage from 30VDC down to 1.5VDC but still achieving high efficiency, 3A output current ensuring current through the blocks of the system.
- High conversion efficiency (up to 94%): Thanks to Buck DC-DC technology, redundant energy is not wasted as thermal energy too much like linear voltage regulator ICs (78xx family), helping the robot's battery last longer and the circuit operate cooler.

Technical specifications:

- Lipo 3S 5400mAh 30C battery:

- Battery type: Lithium Polymer (LiPo).
- Configuration: 3S1P (3 Cells in series).
- Capacity: 5400 mAh (5.4 Ah).

- Rated voltage: 11.1V (3.7V/cell).

- LM2596 3A buck converter circuit:

- Input voltage: From 3V to 30V.
- Output voltage: has a potentiometer adjustable in the range of 1.5V to 30V.
- Switching frequency: 65 KHz.
- Maximum response current: 3A.
- Efficiency: 92%.
- Power: 15W.
- Dimensions: 45×20×14 mm (length×width×height).

- XL4015 DC-DC buck converter module:

- Module type: Non-isolated step-down (Buck Converter).
- Input voltage: 4V – 38V DC.
- Output voltage: 1.25V – 36V DC (Adjustable by potentiometer).
- Maximum output current: 5A (Should be used stably at 4.5A).
- Output power: Maximum 75W.
- Protection features: Integrated over-temperature protection and output short-circuit protection.
- Conversion efficiency: Up to 96%.

Figure 3.6. Components of the Power Supply Module.

3.1.4 Vehicle frame

a. Overview of idea and geometric structure

The vehicle frame system is designed based on the TLWR principle, a breakthrough combination between the stability of wheels and the flexibility of jointed legs. The core goal of this design is to create a skeleton sturdy enough to protect electronic devices, while being light enough to optimize the motor's energy consumption. Based on the actual design image (Figure 3.9), the robot structure is divided into two distinct functional parts:

- Control Chassis: An open box-like structure, reinforced by horizontal braces, to create installation space for the microcontroller, battery, and IMU sensor.
- Multi-linkage Leg System: The robot legs are designed in the form of mechanical linkages, connecting with servo motors to change the height of the center of gravity, helping the robot perform complex movements that a conventional two-wheeled balancing robot cannot do.

b. Design for Additive Manufacturing

Instead of using traditional machining methods such as CNC cutting or Laser, the entire frame was calculated to be manufactured using 3D printing technology. To ensure feasibility and durability, the following factors were integrated directly into the design on SolidWorks:

- Wall Thickness Optimization: The frame plates are designed with a uniform thickness (about 3-5mm) to avoid warping due to thermal stress during the printing process.
- Weight Reduction Structure: Vertical grooves on the body are not only for aesthetics but also help reduce plastic consumption, thereby reducing the robot’s total weight, supporting the balancing system to respond faster.
- Assembly Tolerance Calculation: At the positions for installing bearings, motor shafts, and servos, I have set an offset tolerance from 0.1mm to 0.2mm depending on the printer’s accuracy, ensuring that parts after printing can fit tightly without excessive manual processing.

c. Material selection and 3D printing parameters

The material selected for this project is PLA+ (Polylactic Acid Plus). Reason for choosing PLA+: Has higher stiffness than regular PLA, less shrinkage, and better

impact resistance, very suitable for robot frame parts needing high geometric stability.

Expected technical specifications:

- Infill: 30% - 50% in Hexagon pattern to optimize the ratio between weight and durability.
- Layer Height: 0.2mm to ensure a balance between surface smoothness and machining time.
- Shell count: 3-4 layers to increase rigidity for mechanical screw holes.

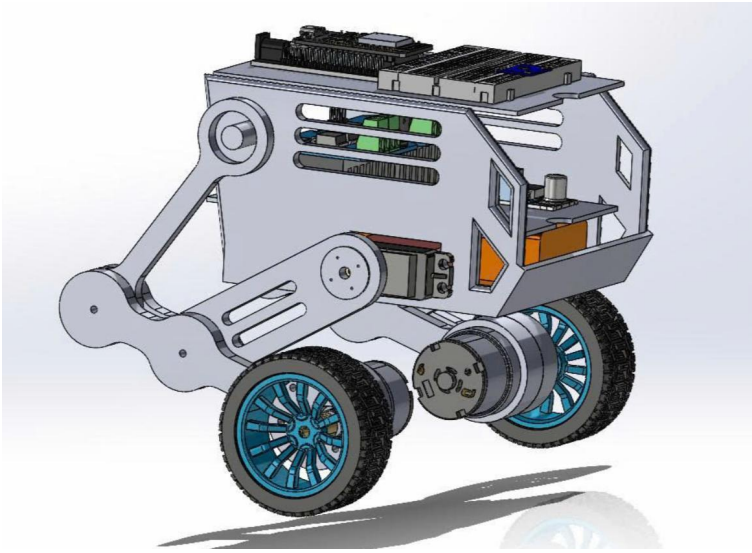


Figure 3.7. Constructing the robot frame using SolidWorks software.

3.2 SIMULATION

3.2.1 MATLAB

In this study, three postures of the TLWR presented in Figure 3.8 are considered in the control design.

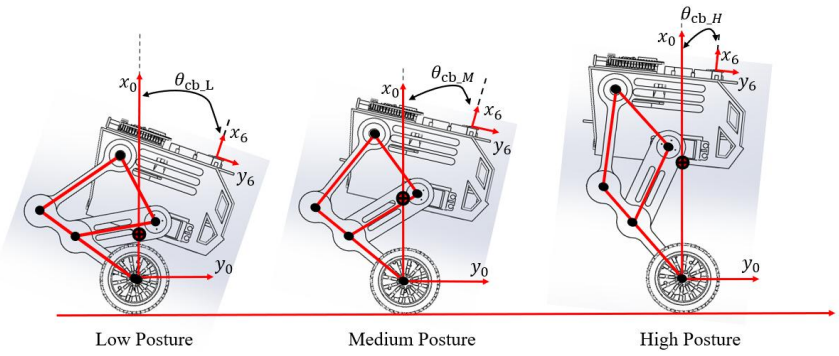


Figure 3.8. Three distinct postures of the TLWR (Two-Wheeled Legged Robot).

a. **Find the control parameter set K for three positions**

i. Find the height L of the upper body center of gravity relative to the wheel axis

After we have established the formula for calculating the center of gravity position in section 2.2, we input the formulas into MATLAB to calculate the height for the three positions; the following is the code when we input it into MATLAB (notated in Appendix 2).

Results:

Case 1: with Low Posture corresponding to the control angle (goc_dk = 115)

⇨ L= 0.0715 (m)

```
goc_dk= 115;
goc_do=goc_dk-37.65;
goc_rad = deg2rad(goc_do);

l25 = sqrt(l2^2 + l4^2 - 2*l2*l4*cos(goc_rad));
```

Command Window

```
========================================
KẾT QUẢ (HỆ QUY CHIẾU: LINK 1)
========================================
Tọa độ X (xc): 0.0574 m
Tọa độ Z (zc): 0.0427 m
----------------------------------------
Khoảng cách từ gốc Link 1 đến CoM tổng: 0.0715 m
========================================
>>
```

Case 2: with Medium Posture corresponding to the control angle (goc_dk = 135)

⇨ L= 0.1319 (m)

```
goc_dk= 135;
goc_do=goc_dk-37.65;
goc_rad = deg2rad(goc_do);

l25 = sqrt(l2^2 + l4^2 - 2*l2*l4*cos(goc_rad));
```

Command Window

```
========================================
KẾT QUẢ (HỆ QUY CHIẾU: LINK 1)
========================================
Tọa độ X (xc): 0.0880 m
Tọa độ Z (zc): 0.0982 m
----------------------------------------
Khoảng cách từ gốc Link 1 đến CoM tổng: 0.1319 m
========================================
>>
```

Case 3: with High Posture corresponding to the control angle (goc_dk = 155)

⇨ L= 0.2000 (m)

```
goc_dk= 155;
goc_do=goc_dk-37.65;
goc_rad = deg2rad(goc_do);

l25 = sqrt(l2^2 + l4^2 - 2*l2*l4*cos(goc_rad));
```

Command Window

```
========================================
KẾT QUẢ (HỆ QUY CHIẾU: LINK 1)
========================================
Tọa độ X (xc): 0.1637 m
Tọa độ Z (zc): 0.1148 m
----------------------------------------
Khoảng cách từ gốc Link 1 đến CoM tổng: 0.2000 m
========================================
>>
```

ii. Find the control parameter set K for three positions

After having found the heights from the wheel axis to the total CoM, we use the formulas in section (2.5.3 LQR Controller Design) in MATLAB to find K; the MATLAB code is established in Appendix 3.

Results:

Case 1: with Low Posture corresponding to L=0.0715 (m)

```
clc;
M = 1.8915;
m = 0.055;
l = 0.0715;
g = 9.81;
r1 = 1;
thetadot_init = 0 ;
theta_init = 0.1;
xdot_init = 0;
x_init = 0;
A = [ 0,              1, 0, 0;
     ((M + m)*g)/(M*l), 0, 0, 0;
      0,              0, 0, 1;
     -(m*g)/M,        0, 0, 0];

B = [ 0;
     -1/(M*l);
      0;
      1/M];

Q = [ 1 0 0 0; 0 1 0 0; 0 0 100 0; 0 0 0 1]
R = r1;
```

Command Window

```
Q =

     1     0     0     0
     0     1     0     0
     0     0   100     0
     0     0     0     1


k =

  -50.5215   -4.3816  -10.0000   -8.0666

>>
```

Case 2: with Medium Posture corresponding to L=0.1319 (m)

```
clc;
M = 1.8915;
m = 0.055;
l = 0.1319;
g = 9.81;
r1 = 1;
thetadot_init = 0 ;
theta_init = 0.1;
xdot_init = 0;
x_init = 0;
A = [ 0,              1, 0, 0;
     ((M + m)*g)/(M*l), 0, 0, 0;
      0,              0, 0, 1;
     -(m*g)/M,        0, 0, 0];

B = [ 0;
     -1/(M*l);
      0;
      1/M];

Q = [ 1 0 0 0; 0 1 0 0; 0 0 100 0; 0 0 0 1]
R = r1;
```

Command Window

```
Q =

     1     0     0     0
     0     1     0     0
     0     0   100     0
     0     0     0     1


k =

  -55.3242   -6.4294  -10.0000   -8.6522

>>
```

Case 3: with High Posture corresponding to L=0.2000 (m)

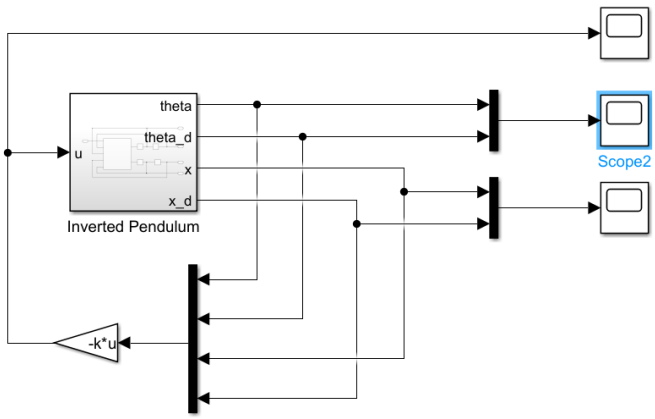
```
clc;
M = 1.8915;
m = 0.055;
l = 0.2;
g = 9.81;
r1 = 1;
thetadot_init = 0 ;
theta_init = 0.1;
xdot_init = 0;
x_init = 0;
A = [ 0,              1, 0, 0;
      ((M + m)*g)/(M*l), 0, 0, 0;
       0,              0, 0, 1;
      -(m*g)/M,        0, 0, 0];

B = [ 0;
     -1/(M*l);
      0;
      1/M];

Q = [ 1 0 0 0; 0 1 0 0; 0 0 100 0; 0 0 0 1]
R = r1;
```

Command Window

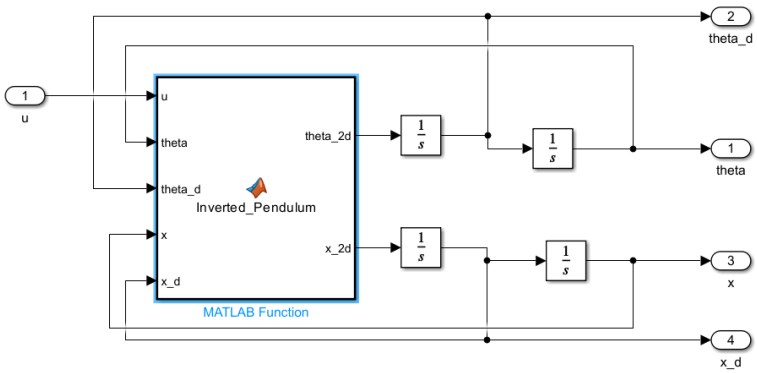
```
Q =

     1     0     0     0
     0     1     0     0
     0     0   100     0
     0     0     0     1


k =

  -59.8829   -8.5306  -10.0000   -9.1736

>>
```

b. **LQR algorithm simulation on Matlab Simulink**

System block diagram:



Inverted pendulum:



MATLAB Function:

```
function [theta_2d,x_2d] = Inverted_Pendulum(u,theta,theta_d,x,x_d)
M = 1.8915;
m = 0.055;
l = 0.2;S
g = 9.81; %m/s^2
%% Con lac nguoc tren xe
x_2d = (u+m*l*sin(theta)*(theta_d)^2-m*g*cos(theta)*sin(theta))/(M+m-m*(cos(theta))^2);
theta_2d = (u*cos(theta)-(M+m)*g*sin(theta)+m*l*(cos(theta)*sin(theta))*(theta_d)^2)/(m*l*(cos(theta))^2-(M+m)*l);
```

3.2.2 ROS/GAZEBO

Simulation on ROS/Gazebo helps to re-verify the LQR control algorithm in an environment that takes into account real physical factors such as friction, gravity, and communication latency, which theoretical simulation on Simulink often ignores.

Step 1: Initialize Workspace and Model Conversion (URDF) In ROS1, every project needs to be organized in a catkin_workspace.

- Create Workspace: Use the catkin_make command to initialize the standard directory structure.

- Export model: Use the URDF Exporter plugin from SolidWorks to convert 3D drawings. The result obtained is a ROS Package containing:

- meshes folder: Contains .stl files describing the 3D-printed vehicle frame shape.
- urdf folder: Contains the .urdf file defining Joints and Links.
- launch folder: Contains XML files for quick environment startup.

Step 2: Configure Gazebo Plugins and Physical Characteristics For the robot to operate in an environment with gravity and friction, the URDF file is supplemented with <gazebo> tags:

- Define powertrain: Use transmission to connect virtual joints with the controller.

- Integrate Sensor Plugin: Use libgazebo_ros_imu.so to simulate the IMU sensor (registering data to the /imu Topic). This is the key component to get tilt angle data for the LQR algorithm.

Step 3: Design Control Nodes using Python/C++ The system is operated based on the communication mechanism between Nodes through the ROS Master:

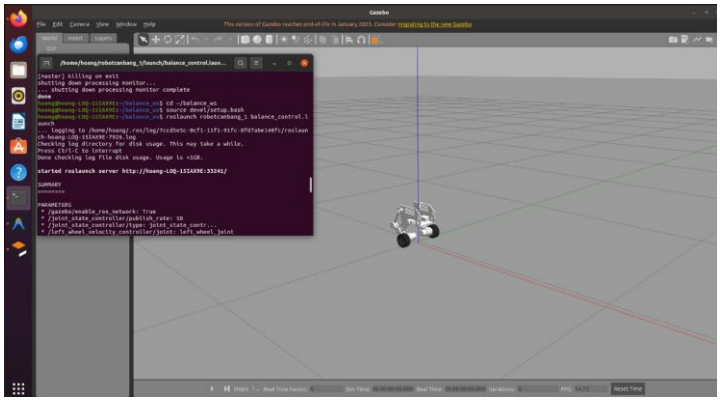
- LQR_Controller Node: This is the central Node, executing the calculated K gain matrix from MATLAB. This Node will perform:

- Synchronization: The control frequency is set fixed using the rospy.Rate() function to ensure real-time stability for the balancing system.

Step 4: Run Launch file and Data Visualization

- Launch: Use the command roslaunch [package_name] [file_name].launch. This command will simultaneously open Gazebo and spawn the robot at coordinates (0,0,0).

- Check response: Use the rqt_plot tool to draw real-time tilt angle charts, thereby evaluating the robot's self-balancing ability when there are disturbances in Gazebo.



CHAPTER 4: ACTUAL MODEL DESIGN

In this chapter, the control algorithms of the system are presented, presented in the form of flowcharts and explaining the use of algorithms according to the set design requirements. Then, introducing the monitoring software interface designed using the C# language.

4.1 Design requirements

Regarding the system control software, the design requirements to ensure the vehicle balances are as follows:

- Read signals from the MPU6050 sensor via I2C communication.
- Read the returned encoder pulse values.
- Output PWM pulses to control the H-bridge circuit to control the motor according to speed, forward rotation, reverse rotation, and motor stop.
- Limit the tilt angle for vehicle balancing from -30° to 30°.
- Communicate with the computer via the USB port.
- Design an interface to draw response charts when controlling the balancing vehicle in real-time.

4.2 Building the control algorithm

To use and program the ESP32, the software to use is Arduino IDE 1.8.15, convenient for connecting the ESP32 to the computer with a USB cable. The programming language of Arduino is C/C++, which is much simpler than other types of microcontrollers thanks to a large user community and diverse available programming support libraries. The control program is included in Appendix 1.

4.2.1 Main program algorithm

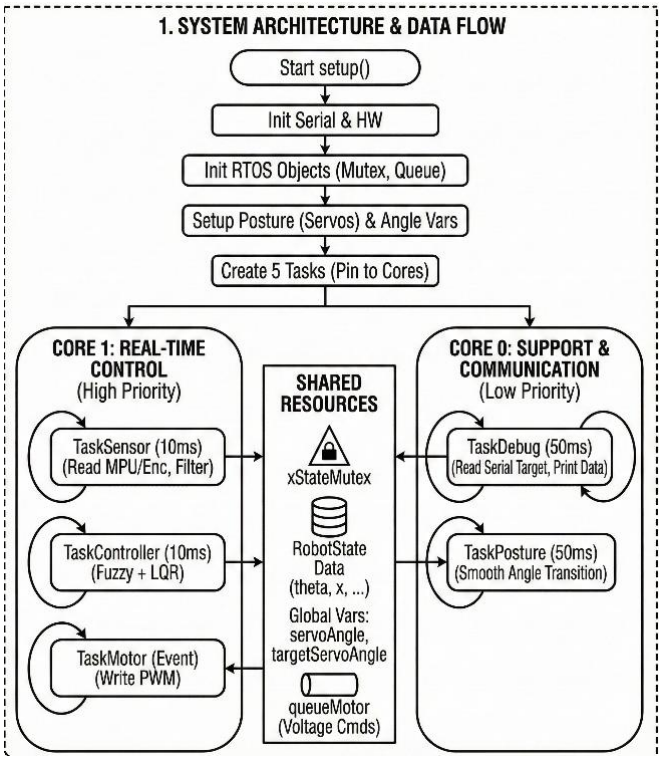


Figure 4.1. Flowchart of the main program algorithm.

The main program plays a central role in initializing the hardware, setting up the Real-Time Operating System (FreeRTOS) environment, and coordinating parallel processing threads. Due to the characteristics of self-balancing robots requiring fast response speeds and low latency, the software architecture is designed to leverage the multi-threading processing power of the ESP32 microcontroller.

a. System Initialization Stage As soon as the system is powered on, the setup() function is executed first to set basic configurations:

b. Initialize RTOS Resources: To ensure a safe communication mechanism between parallel running tasks, the program proceeds to initialize FreeRTOS coordination objects.

c. Task Allocation & Pinning: Instead of using a sequential single-threaded loop() function, the main program creates 5 independent Tasks and pins them fixed to the 2 CPU cores to optimize performance:

i. Real-time Task Group (Pinned to Core 1): This is the group of highest priority tasks (High Priority), directly responsible for the robot's stability: TaskSensor, TaskController, TaskMotor.

ii. Support Task Group (Pinned to Core 0): The lower priority task group (Low Priority), performing tasks that do not have strict time requirements: TaskDebug, TaskPosture.

d. Scheduler Execution: After all resources and tasks have been successfully initialized, the program calls the vTaskStartScheduler() function. At this point, the FreeRTOS kernel will take control, automatically dividing CPU time for tasks based on priorities and interrupt events. The default Arduino loop() function will be disabled or left empty because the main processing flow has been moved to RTOS tasks.

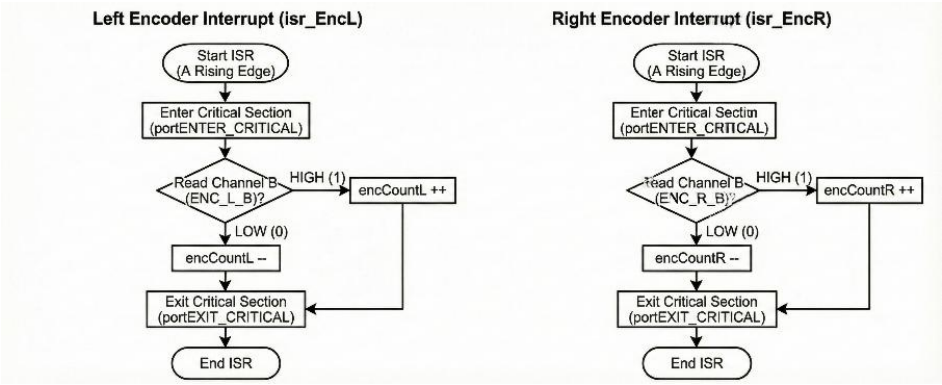


Figure 4.2. Flowchart of the interrupt service routine (ISR) for processing encoder pulses.

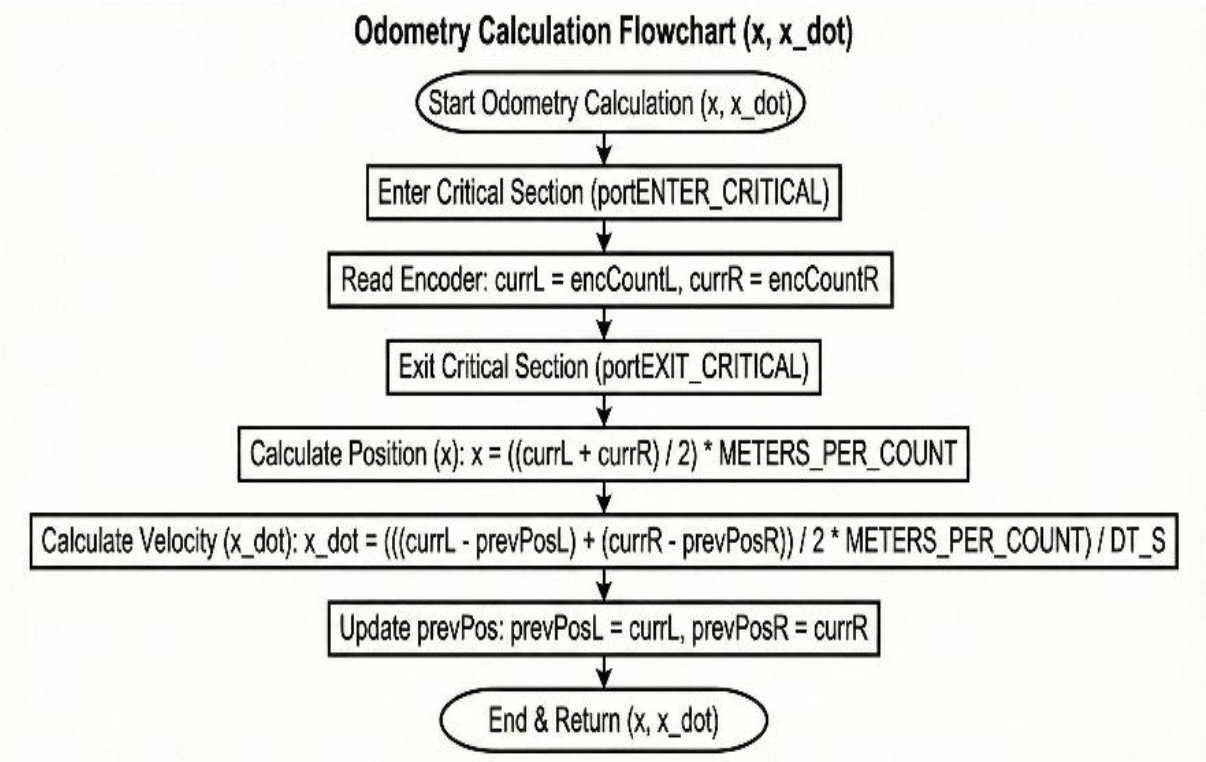


Figure 4.3. Flowchart for calculating displacement and linear velocity.

Initialization part:

- PIN 34 connects to encoder channel A of the left motor (using interrupt 0).
- PIN 35 connects to encoder channel B of the left motor (reading feedback signal).
- PIN 39 connects to encoder channel A of the right motor (using interrupt 1).
- PIN 36 connects to encoder channel B of the right motor (reading feedback signal).

Algorithm for reading left motor values:

- Use interrupt 0 for channel A.
- When a rising edge pulse occurs, the interrupt program executes.
- At this time, read the feedback value from channel B: if channel B is at a high level, the motor is rotating forward, leftencoder++; conversely, if channel B is at a low level, the motor is rotating in the reverse direction, leftencoder--.

Algorithm for reading right motor values:

- Use interrupt 1 for channel A.
- When a rising edge pulse occurs, the interrupt program executes.

- At this time, read the feedback value from channel B: if channel B is at a high level, the motor is rotating forward, rightencoder++; conversely, if channel B is at a low level, the motor is rotating in the reverse direction, rightencoder--.

Using this method will optimize the processing speed of the microcontroller instead of the method using interrupts for both channels A and B.

Algorithm for reading tilt angle and rotation angle:

- From the construction of the object mathematical model in Chapter 2, we obtain the formula for calculating the average tilt angle of the two wheels from the equation.

- The motor uses a gearbox with a reduction ratio of 34:1, and the Encoder disk returns 11 pulses/revolution. Therefore, the number of Encoder pulses per 1 revolution of the main motor shaft is 30 x 11 = 330 pulses/revolution.

The Encoder angle is determined by the formula:

$$\text{Góc Encoder} = \frac{\text{Xung encoder đọc được} \times 360^{o}}{\text{Xung encoder khi xoay 1 vòng}} = \frac{\text{Xung encoder đọc được} \times 360^{o}}{330}$$

Suy ra:

$$\begin{bmatrix} \theta \\ \phi \end{bmatrix} = \begin{bmatrix} \left(\frac{1}{2} \times \frac{360}{317}\right)(leftencoder + rightencoder) \\ \left(\frac{R}{W} \times \frac{360}{317}\right)(leftencoder + rightencoder) \end{bmatrix} \quad (0.1)$$

With: $leftencoder$ is the encoder pulses read from the left wheel.

$rightencoder$ is the encoder pulses read from the right wheel.

4.2.2 Algorithm for reading and processing values from the tilt angle sensor

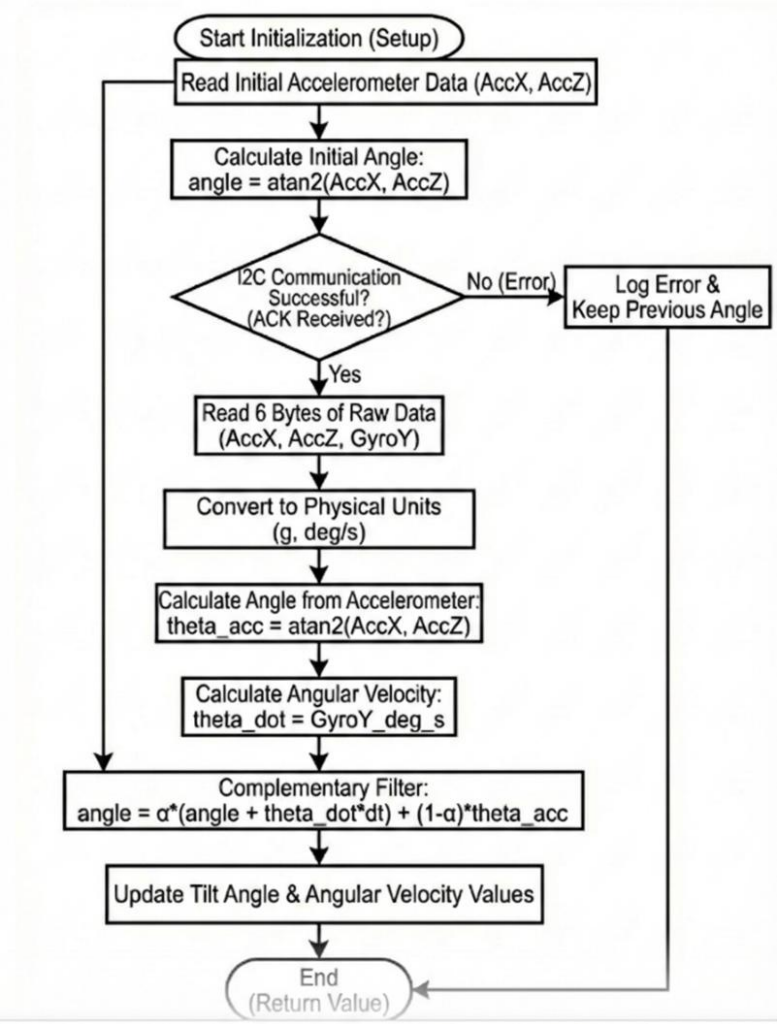


Figure 4.4. Flowchart of the algorithm for reading and processing tilt sensor data.

The process of determining the robot's tilt angle is a prerequisite for maintaining balance. The MPU6050 sensor reading flowchart is executed according to a fixed sampling cycle (Sampling Time) to ensure real-time performance for the digital filter.

a. Initialization and Verification: Before entering the main loop, the system performs an initial reading of static acceleration values to determine the starting tilt angle, helping the filter converge faster upon startup.

b. Data Collection: At each cycle, the microcontroller sends a request via I2C communication. If an acknowledgment signal (ACK) is received, the system reads 6 bytes of raw data, including 3-axis acceleration (Acc_x, Acc_z) and Y-axis angular velocity (Gyro_y).

c. Primary angle calculation:

i. The angle from the accelerometer (θ_{acc}) is calculated using the atan2 function based on the gravity vector, providing an accurate absolute value but with high noise during vibrations.

ii. The angular velocity($\dot{\theta}$) is taken directly from the gyroscope after unit conversion.

Sensor Fusion: The system utilizes a Complementary Filter or Kalman filter to combine the advantages of two data sources: the long-term stability of the accelerometer and the instantaneous sensitivity of the gyroscope. The complementary filter formula: The final result is a smooth tilt angle(θ) and angular velocity($\dot{\theta}$) which are updated into the robotState state variable.

4.2.3 PWM pulse width modulation algorithm for each wheel using LQR and motor control

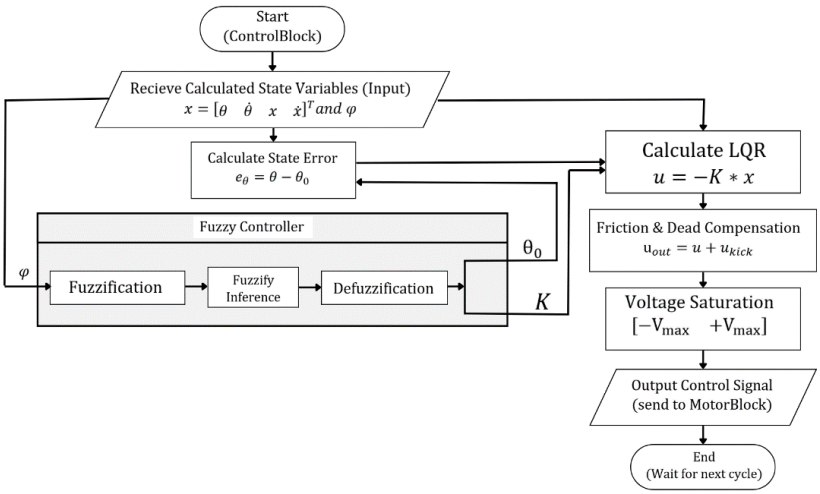


Figure 4.5. Flowchart of the PWM calculation algorithm for each wheel using the LQR controller.

PWM Pulse Width Modulation Algorithm for each wheel:

- TaskController receives the state variables $x = [\theta \quad \dot{\theta} \quad x \quad \dot{x}]$ And the servo joint angle value φ is the input value.

- Fuzzy Block: Receives the servo joint angle φ. Through the process of fuzzification, applying fuzzy rules, and defuzzification, the Fuzzy Controller decides the appropriate control parameter set K and the mechanical offset angle θ_0 for the specific posture being considered.

- Since the feedback angle value θ for balance at each posture is different, it is necessary to redefine the deviation angle relative to the equilibrium position using the formula: $e_\theta = \theta - \theta_0$.

- From the K matrix calculated in the theoretical foundation chapter, the K matrix will be in the form:

$$K = \begin{bmatrix} K_1 & K_2 & K_3 & K_4 \\ K_1 & K_2 & K_3 & K_4 \end{bmatrix} \tag{0.2}$$

And the control signal:

$$u = \begin{bmatrix} v_l \\ v_r \end{bmatrix} \tag{0.3}$$

According to the LQR algorithm, the control signal supplied to the motors is:

$$u = -Kx(t) \tag{0.4}$$

Consequently, the control signals for the left and right wheels are respectively:

$$\begin{cases} u_R = K_1 * e_\theta + K_2 * \dot{\theta} + K_3 * x + K_4 * \dot{x} \\ u_L = K_1 * e_\theta + K_2 * \dot{\theta} + K_3 * x + K_4 * \dot{x} \end{cases} \tag{0.5}$$

- In practice, however, the motor does not start from 0V; it requires a certain voltage value to overcome static friction. Therefore, we add a constant offset value to ensure the control values are accurate.

$$u_{out} = u + u_{kick}$$

- The voltage values are then sent to the TaskMotor to control the motors.

4.2.4 Output and plotting charts on the computer

Establish RS232 serial communication via the USB programming port with a baud rate of 115200 using the syntax Serial.begin(115200).

Read the system's theta, psi, and phi values from the sensors and encoders, then store them in memory variables.

Output the values to the serial port using the function Serial.println((String)(psi * ToDeg) + ',' + (String)(theta * ToDeg) + ',' + (String)(phi * ToDeg)); to send a string of characters to the computer, separated by commas.

Design the interface using the C# programming language; the read values are separated by commas to facilitate string splitting and parsing into double types when data is uploaded to the computer. The ZedGraph library is utilized to create the line chart plotting framework.

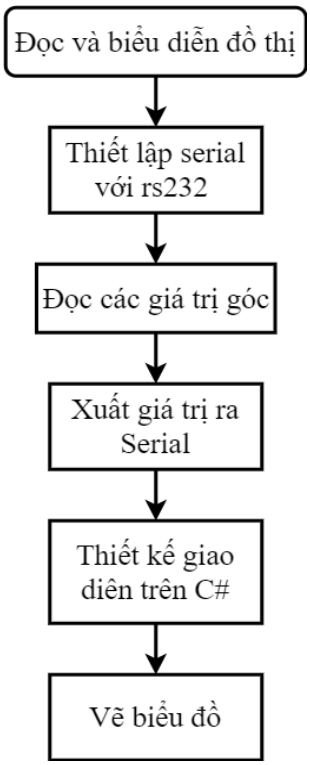


Figure 4.6. Flowchart of the data acquisition and real-time plotting algorithm.

4.3 Monitoring Software

The monitoring software is designed with an interface consisting of:

- Terminal Connection Area: Includes options to select the Baudrate and Port of the microcontroller, a Connect button, a Disconnect button, and an Exit button to close the program.

- Received Signal Area: Displays the feedback signals for the tilt angle ψ (Psi) in real-time.

- Charting Area: This section plots the real-time feedback tilt angle, with the vertical axis representing the tilt angle ψ (degrees), the horizontal axis represents time (seconds).

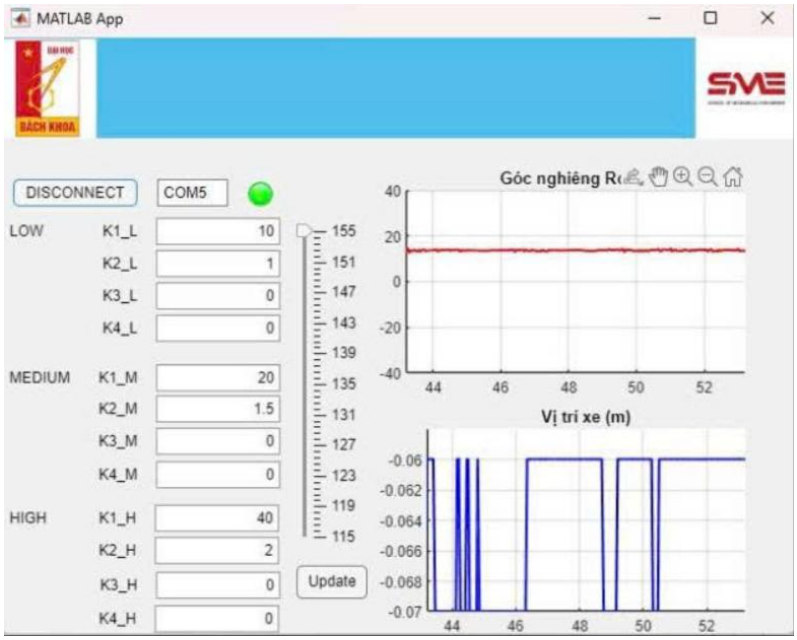


Figure 4.7. Graphical User Interface (GUI) for system monitoring.

CHAPTER 5: CONCLUSION AND FUTURE PERSPECTIVES

In this chapter, the team presents the simulation results and the actual performance achieved, followed by specific evaluations for each case. Based on these results, the limitations and shortcomings encountered during the project are identified, and potential directions for future development are proposed.

5.1 Achieved Results

5.1.1 Model Design and Simulation Results

- Hardware Connection Diagram and General Specifications

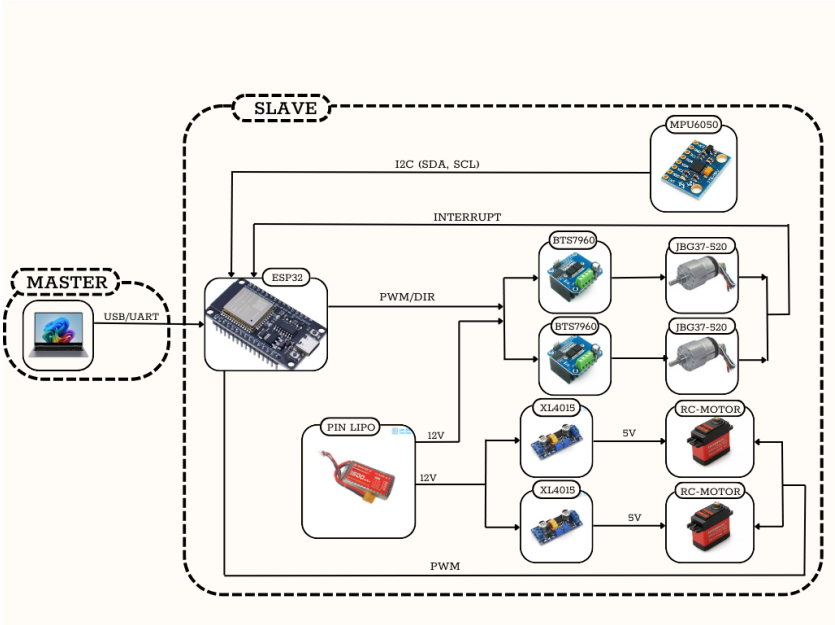


Figure 5.1. Hardware Connection Diagram.

Table 5.1. Parameters of the two-legged wheeled robot

Symbol	Value	Unit	Symbol	Value	Symbol
a	0.0438	m	l_2	0.080	m
r	0.0325	m	l_3	0.0954	m
d	0.1848	m	l_4	0.0742	m
m_1	0.2051	kg	l_5	0.1030	m
m_2	0.0548	kg	l_{c1}	0.0593	m
m_3	0.0672	kg	l_{c2}	0.0459	m
m_4	1.6072	kg	l_{c3}	0.0477	m
l_1	0.1155	m	l_{c4}	0.0044	m

Table 5.2. Parameters of the two-legged wheeled robot

Posture	Height	Target Angle	Parameters
Low	0.0715	12.5^o	$K_1 = \begin{bmatrix} 72.6 & 10.2 & 8.5 & 2 \\ 72.6 & 10.2 & 8.5 & 2 \end{bmatrix}$
Medium	0.1319	13.2^o	$K_2 = \begin{bmatrix} 81.3 & 10.6 & 8.4 & 2.2 \\ 81.3 & 10.6 & 8.4 & 2.2 \end{bmatrix}$
Heigh	0.2000	13.6^o	$K_3 = \begin{bmatrix} 89.4 & 13.5 & 7.8 & 2 \\ 89.4 & 13.5 & 7.8 & 2 \end{bmatrix}$

- Mechanical Design in SolidWorks:

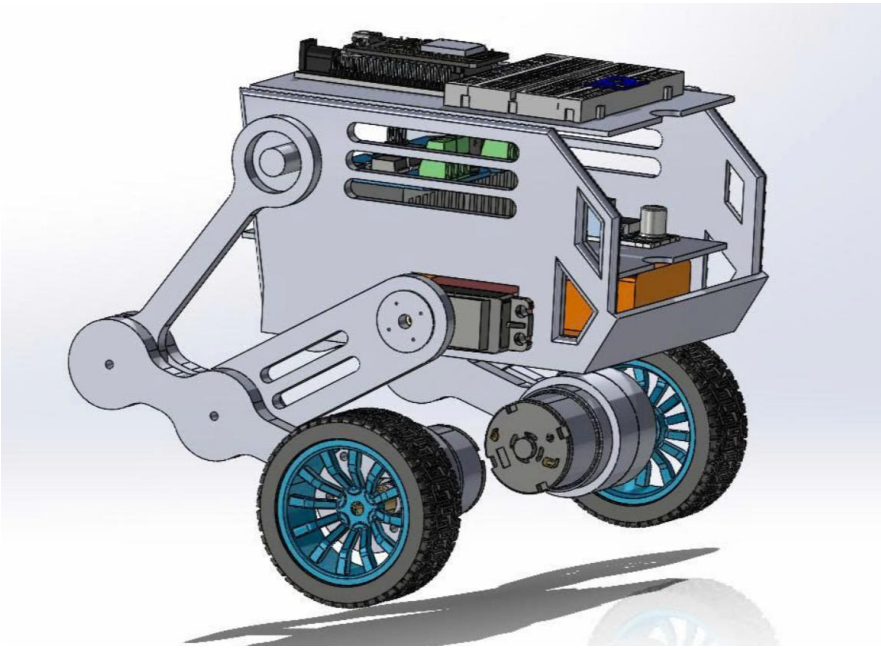


Figure 5.2. Physical model of the Two-Wheeled Balancing Robot (TWBR).

- Simulation results on MATLAB Simulink:

Results:

Case 1: Low Posture

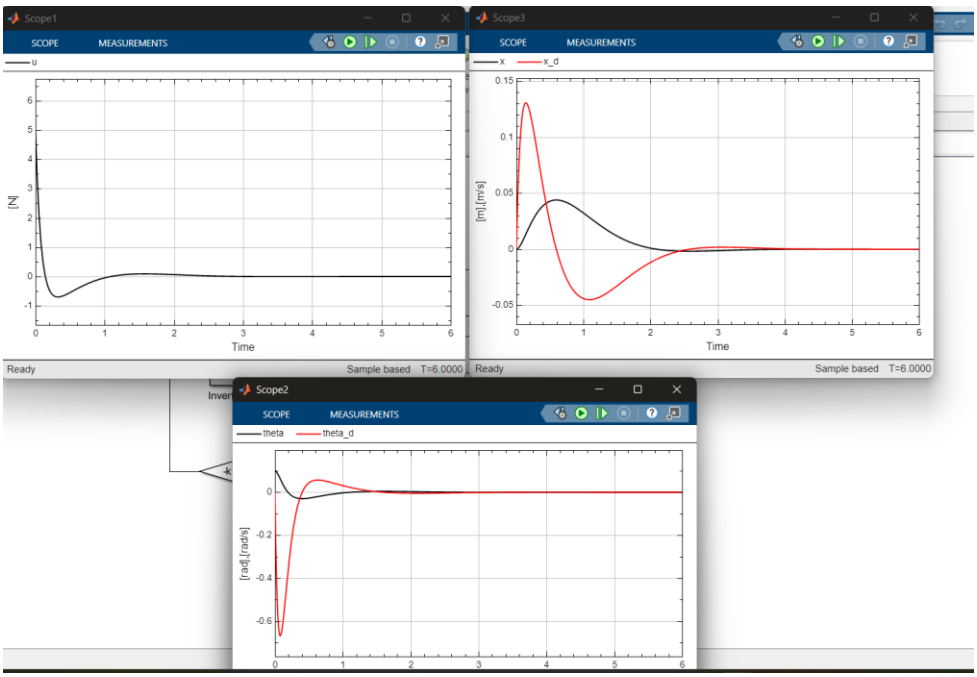


Figure 5.3. Response of the pitch angle, tilt angle, and yaw angle in Case 1.

Case 2: Medium Posture

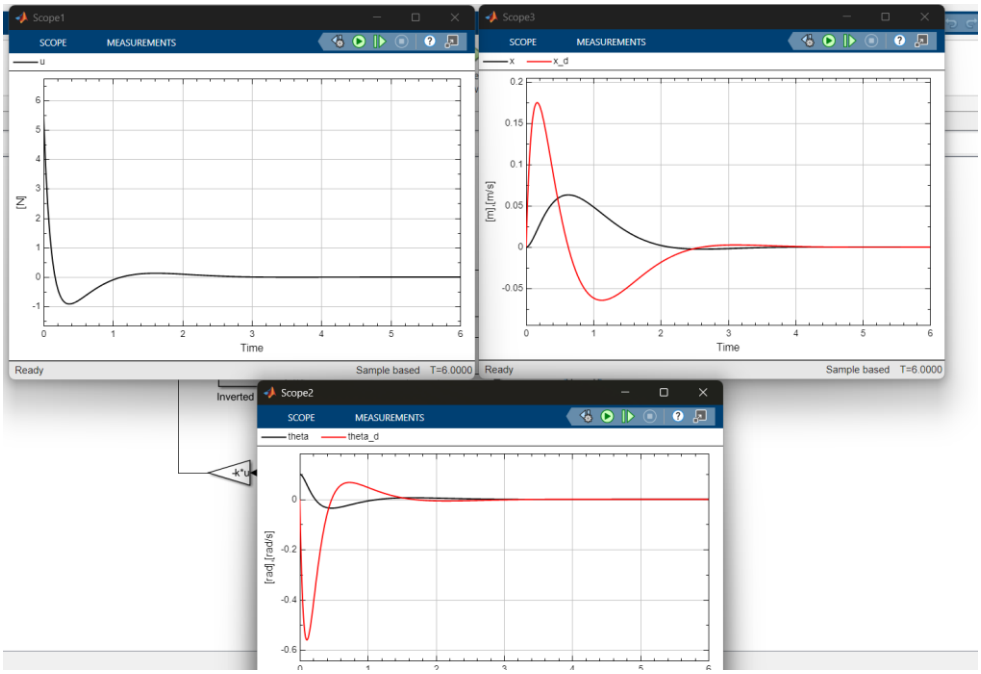


Figure 5.4. Response of the pitch angle, tilt angle, and yaw angle in Case 2.

Case 3: High Posture

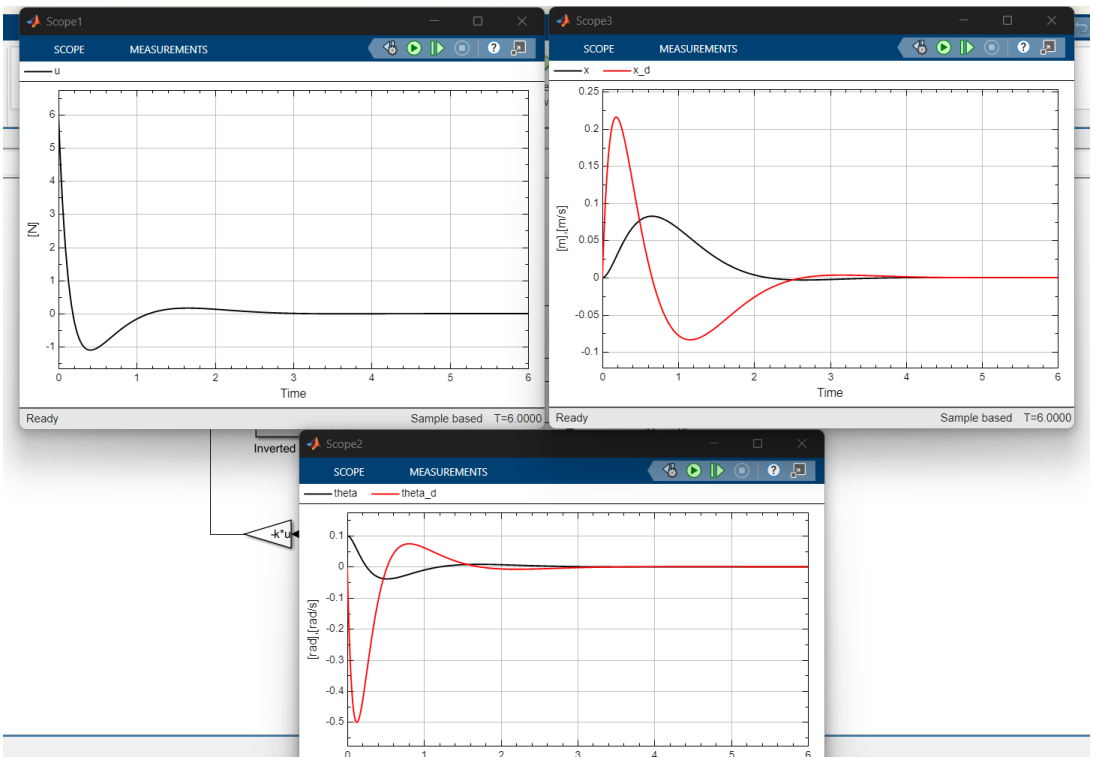


Figure 5.5. Response of the pitch angle, tilt angle, and yaw angle in Case 3.

- Simulation results on ROS/GAZEBO:

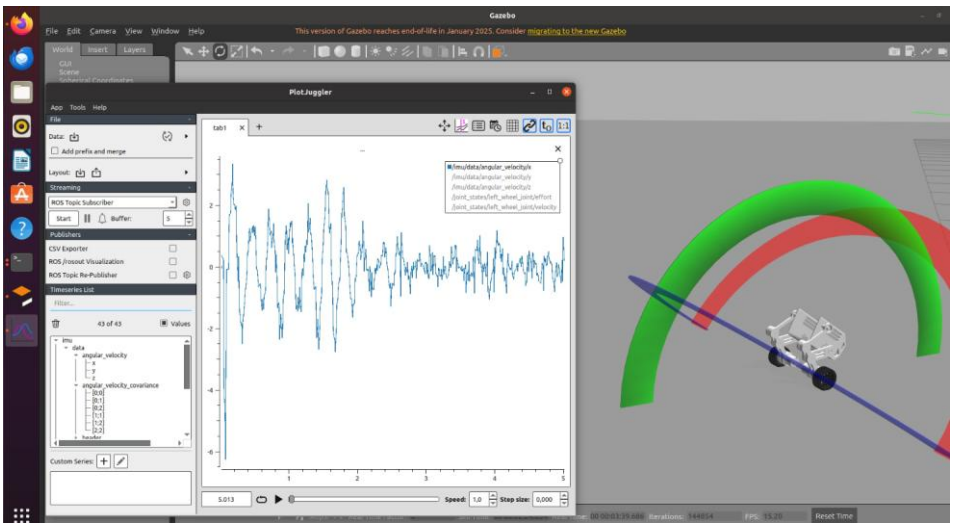


Figure 5.6 The robot's equilibrium state on Gazebo

5.1.2 Experimental Results

- Hardware Prototype:

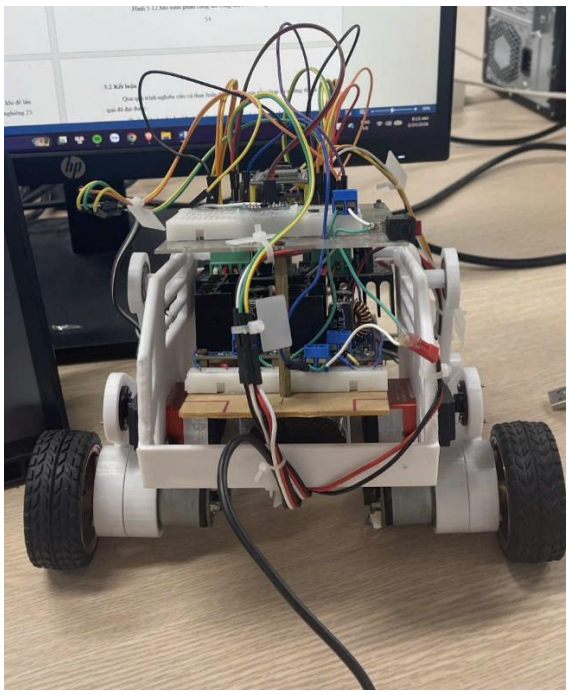


Figure 5.7. Physical prototype - Front view.

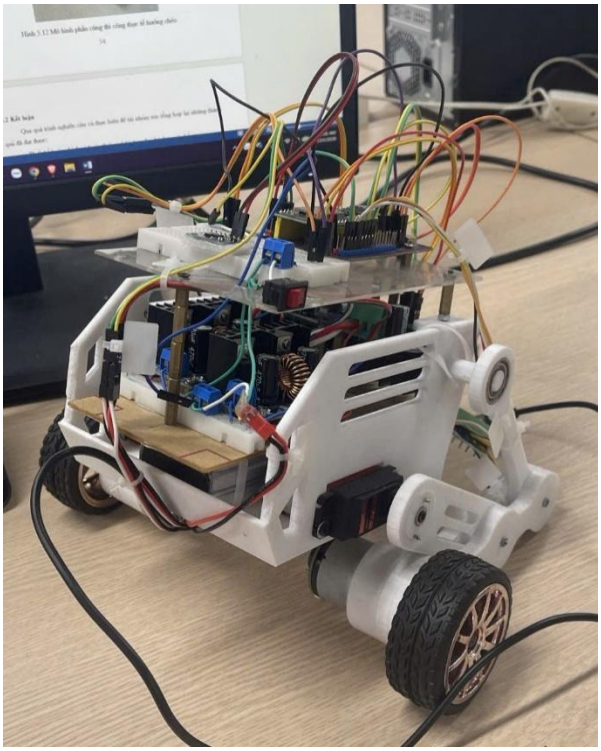


Figure 5.8. Physical prototype - Isometric/Perspective view.

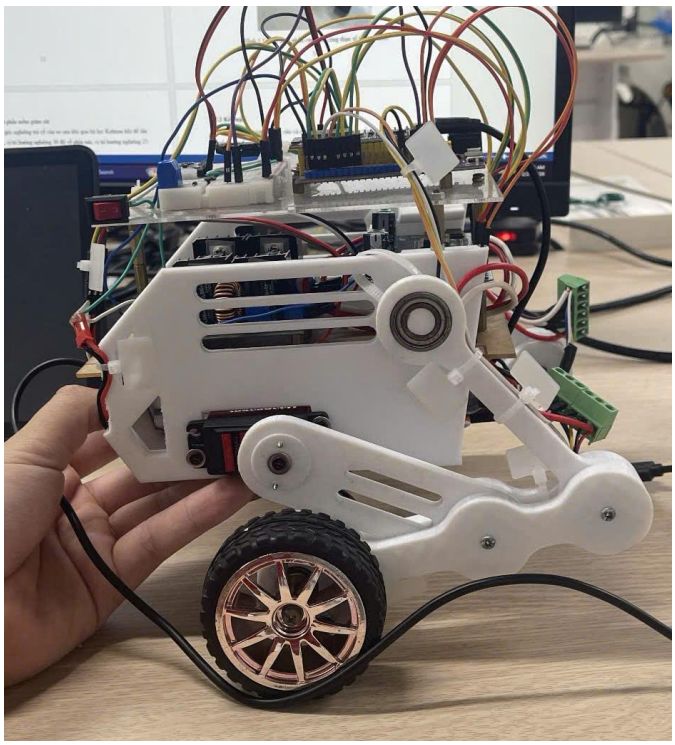


Figure 5.9. Physical prototype - Side view in Low Posture.

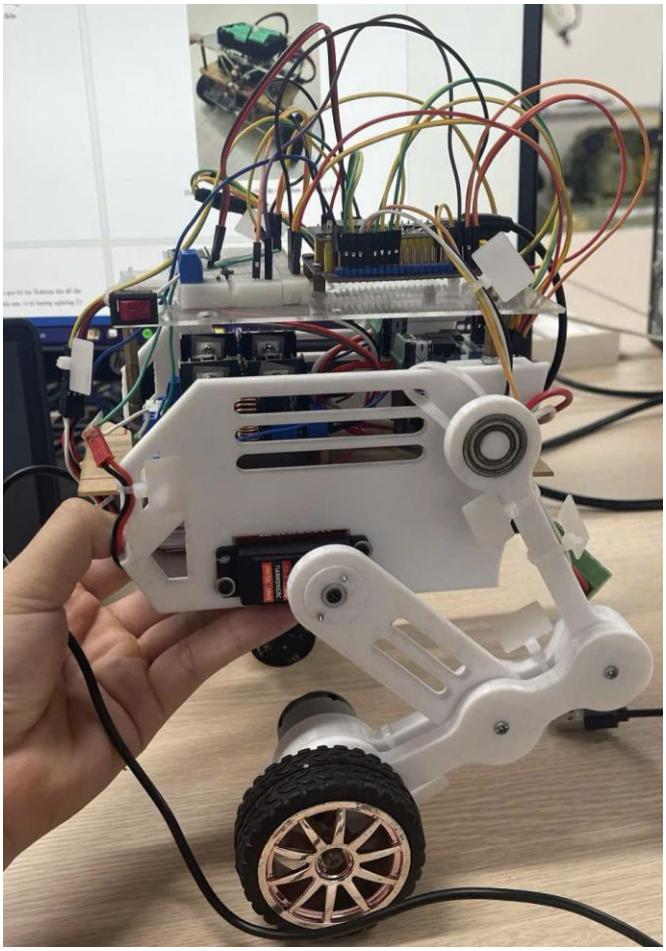


Figure 5.10. Physical prototype - Side view in Medium Posture.

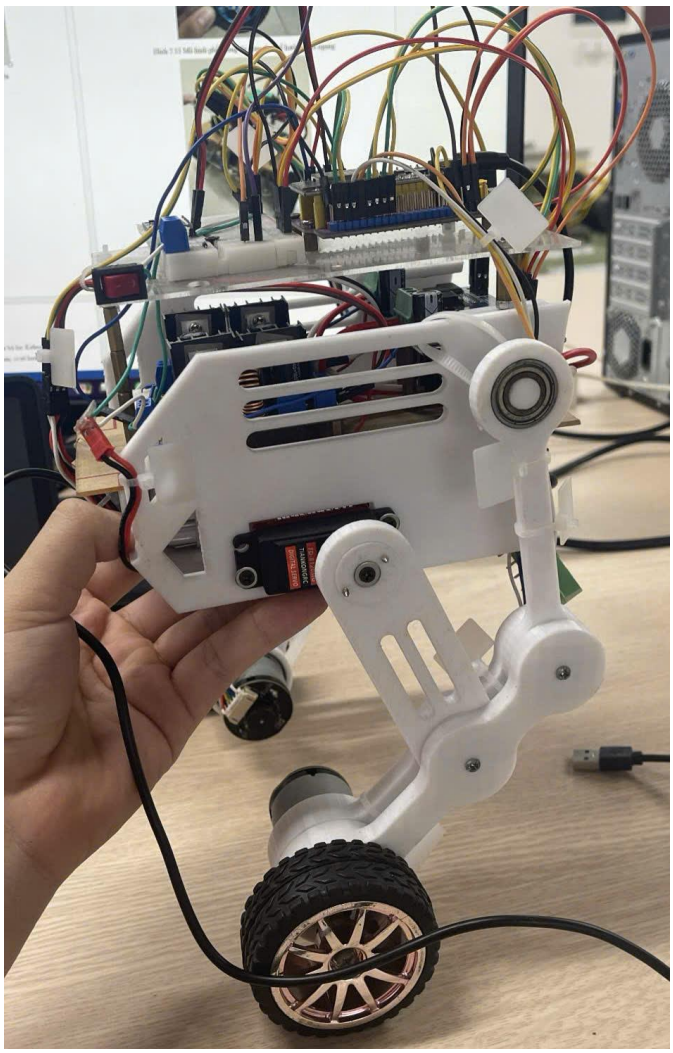


Figure 5.11. Physical prototype - Side view in High Posture.

- Monitoring Software Results:

When the vehicle is balanced, the tilt angle of vehicle ψ is returned to the correct position and displayed on the monitoring interface.

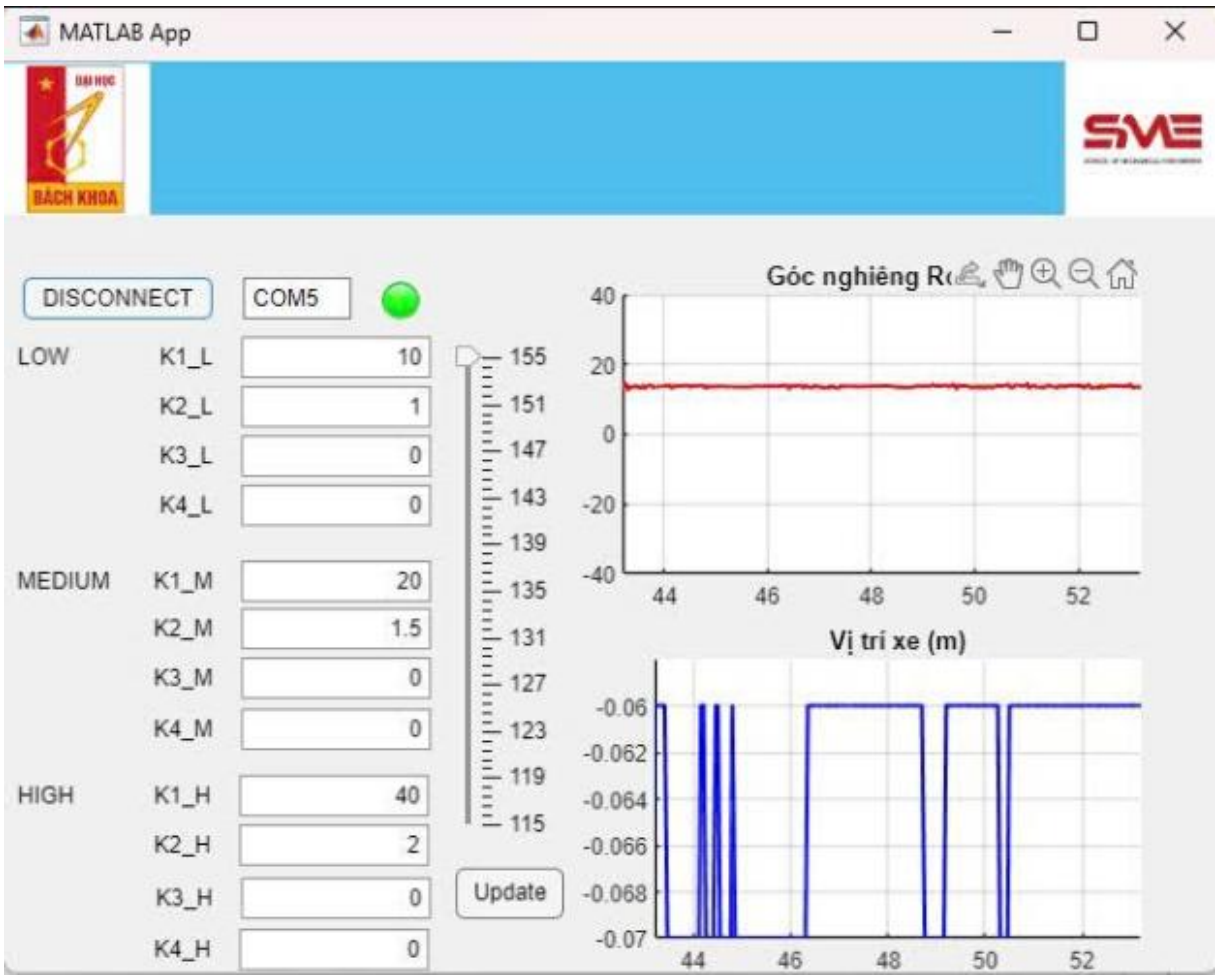


Figure 5.12 Shows the tilt angle and return position of the vehicle during operation.

5.2 Conclusion

Through the research and implementation of this project, the group would like to summarize the achievements:

- Complete and refined design of the self-balancing two-wheeled vehicle model.
- Calculation of the mathematical model and state function for the system.
- Successful simulation of the system on MATLAB Simulink.
- Electrical components and microcontroller function well:

  - Relatively accurate return values from the tilt angle sensor, applying a Kalman filter for good noise reduction.
  - Good pulse width modulation for the H-bridge circuit, stable and not overloaded or overheating during operation.
  - Relatively accurate reading of the rotary encoder.

- Stable microcontroller operation and fast signal processing.

- Communication with the computer via UART and design of monitoring software to plot the vehicle's operation graph.

- The model can balance in place.

However, due to limited time, the project still has many limitations.

5.3 Limitations of the Project

The limitations encountered during the research process are:

- Many different terrains such as rough, undulating, inclined, rough or highly adhesive surfaces, etc., have not been specifically surveyed due to the complexity of the terrain.

- The actual vehicle model still has many errors compared to the simulation, such as moment of inertia, gearbox play on the wheel motor, etc., and the motor used has not yet fully identified all parameters, leading to inaccurate modeling and LQR matrix calculations for the actual model. Therefore, there is still a significant difference between actual control and simulation.

- The robot lacks the ability to self-control or make decisions such as self-movement, obstacle avoidance, self-standing, and energy management to stabilize control over a long period.

- While maintaining balance is quite good, there is still a high overshoot of the robot's tilt angle component.

- The ability to withstand external forces is still weak.

- It can only maintain balance in place; it cannot move.

- The system does not respond well when the coefficient of friction or the mass M of the vehicle changes.

5.4 Future Development

Based on the limitations mentioned above, the group proposes several future development directions for the self-balancing two-wheeled vehicle system as follows:

- Investigate and build a complete kinematic characteristic of the object model under different terrain conditions.

- Develop an intelligent algorithm to identify parameters that have not yet been modeled as mentioned above.

- Investigate and build the ability to stand up on its own, the ability to move autonomously, etc., to help the robot operate more independently and have more practical applications.
- The model will be controlled via Bluetooth connection to a computer or phone for convenient remote control.
- The interface will be designed to be user-friendly and easy to use with more functions.
- Optimizing the vehicle's weight and control algorithms allows for smooth, balanced operation and flexibility across all terrains, making it easy to control and energy-efficient.
- Further development of cameras and GPS for positioning, recording, and image processing enables the robot to perform more applications in larger workspaces.
- This model serves as a foundation for developing self-balancing two-wheeled vehicles capable of carrying people or objects more flexibly and with greater load capacity.

REFERENCES

[1] J. Fang, "The LQR Controller Design of Two-Wheeled Self-Balancing Robot Based on the Particle Swarm Optimization Algorithm," Mathematical Problems in Engineering, vol. 2014, p. 729095, 2014, doi: 10.1155/2014/729095..

[2] C. Xu, M. Li, and F. Pan, "The system design and LQR control of a two-wheels self-balancing mobile robot," in 2011 International Conference on Electrical and ControlEngineering,pp. 2786-2789, 2011, doi: 10.1109/ICECENG.2011.6057680.

[3] S. Srinivas, S. Ramachandiran, and S. Rajendran, "Autonomous robot driven deliveries: A review of recent developments and future directions," Transportation Research Part E: Logistics and Transportation Review, vol. 165, p. 102834, 2022.

[4] C. H. G. Li, L. P. Zhou, and Y. H. Chao, "Self-Balancing Two-Wheeled Robot Featuring Intelligent End-to-End Deep Visual-Steering," IEEE/ASME Transactions on Mechatronics, vol. 26, no. 5, pp. 2263 2273, 2021, doi: 10.1109/TMECH.2020.3036579.

[5] W. Bauer, M. Bender, M. Braun, P. Rally, and O. Scholtz, "Lightweight robots in manual assembly–best to start simply," Frauenhofer-Institut für Arbeitswirtschaft und Organisation IAO, Stuttgart, vol. 1, 2016.

[6] H. W. Kim and S. Jung, "Fuzzy logic application to a two-wheel mobile robot for balancing control performance," International Journal of Fuzzy Logic and Intelligent Systems, vol. 12, no. 2, pp. 154-161, 2012.

[7] S. Kim and S. Kwon, "Nonlinear Optimal Control Design for Underactuated Two-Wheeled Inverted Pendulum Mobile Platform," IEEE/ASME Transactions on Mechatronics, vol. 22, no. 6, pp. 2803 2808, 2017, doi: 10.1109/TMECH.2017.2767085.

[8] K. V. Chate García, O. E. Prado Ramírez, and C. F. Rengifo Rodas, "Comparative analysis between fuzzy logic control, lqr control with kalman filter and pid control for a two wheeled inverted pendulum," in Advances in Automation and Robotics Research in Latin America: Proceedings of the 1st Latin American Congress on Automation and Robotics, pp. 144-156, 2017.

[9] A. A. Bature, S. Buyamin, M. N. Ahmad, and M. Muhammad, "A comparison of controllers for balancing two wheeled inverted pendulum robot," International

Journal of Mechanical & Mechatronics Engineering, vol. 14, no. 3, pp. 62-68, 2014.

[10] N. G. M. Thao, D. H. Nghia, and N. H. Phuc, "A PID backstepping controller for two-wheeled self-balancing robot," International Forum on Strategic Technology 2010, pp. 76-81, 2010, doi: 10.1109/IFOST.2010.5668001.

[11] T. Nikita and K. T. Prajwal, "PID Controller Based Two Wheeled Self Balancing Robot," 2021 5th International Conference on Trends in Electronics and Informatics (ICOEI), pp. 1-4, 2021, doi: 10.1109/ICOEI51242.2021.9453091.

[12] S. Sekiguchi, A. Yorozu, K. Kuno, M. Okada, Y. Watanabe, and M. Takahashi, "Human-friendly control system design for two-wheeled service robot with optimal control approach," Robotics and Autonomous Systems, vol. 131, p. 103562, 2020.

APPENDICES

Appendix 1: Arduino Source Code for the Two-Wheeled Self-Balancing Robot System

```
----------------------------------------1. WRB.ino-----------------------------------------
#include <Arduino.h>
#include <freertos/FreeRTOS.h>

// Include các file Header
#include "SensorBlock.h"
#include "PostureBlock.h"
#include "ControlBlock.h"
#include "MotorBlock.h"
#include "DebugBlock.h"

// --- KHAI BÁO BIẾN TOÀN CỤC ---
SemaphoreHandle_t xStateMutex; // Khóa bảo vệ dữ liệu
QueueHandle_t queueMotor;      // Hàng đợi lệnh Motor
RobotState robotState;         // Trạng thái Robot

void setup() {
  Serial.begin(115200);
  Serial.println("--- WRB SYSTEM STARTING ---");

  // 1. Khởi tạo RTOS
  xStateMutex = xSemaphoreCreateMutex();
  queueMotor = xQueueCreate(1, sizeof(MotorCommand));

  // 2. Cài đặt Servo (Posture)
  setupPosture();

  // 3. Khởi tạo giá trị ban đầu
```

```
  robotState.height = 0.16;
  robotState.theta = 0.0;
  robotState.x = 0.0;
  robotState.theta_dot = 0.0;
  robotState.x_dot = 0.0;

  // 4. TẠO TASK (ĐÃ BẬT TẤT CẢ)

  // Core 1 (Tính toán & Điều khiển)
  xTaskCreatePinnedToCore(TaskSensor, "Sens", 4096, NULL, 4, NULL, 1);
  xTaskCreatePinnedToCore(TaskController, "Ctrl", 4096, NULL, 3, NULL, 1);
  xTaskCreatePinnedToCore(TaskMotor, "Mot", 2048, NULL, 2, NULL, 1);

  // Core 0 (Phụ trợ & Debug)
  xTaskCreatePinnedToCore(TaskDebug, "Dbg", 2048, NULL, 1, NULL, 0);
  xTaskCreatePinnedToCore(TaskPosture, "Post", 2048, NULL, 1, NULL, 0);
}

void loop() {
  vTaskDelete(NULL);
}
```

----------2. SensorBlock.h----------

```
#ifndef SENSOR_BLOCK_H
#define SENSOR_BLOCK_H

#include <Arduino.h>
#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>

// Cấu hình phần cứng
```

```
#define MPU_ADDR 0x68
#define ENC_L_A 34
#define ENC_L_B 35
#define ENC_R_A 39
#define ENC_R_B 36

// Struct dữ liệu
struct RobotState {
  float theta;
  float theta_dot;
  float x;
  float x_dot;
  float height;
  float u_out; // Tín hiệu điều khiển (để debug)
};

extern SemaphoreHandle_t xStateMutex;
extern RobotState robotState;

void TaskSensor(void *pvParameters);
#endif
```

----------3. SensorBlock.cpp----------

```
#include "SensorBlock.h"
#include <Wire.h>

#define ALPHA 0.98
#define ALPHA_GYRO 0.7
#define DT_MS 10
#define DT_S 0.01
```

```
// Thông số cơ khí: R=0.0325m, Encoder 11 xung, Hộp số 30
const float METERS_PER_COUNT = (2 * PI * 0.0325) / (11.0 * 30.0);

volatile long encCountL = 0;
volatile long encCountR = 0;
portMUX_TYPE muxEnc = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR isr_EncL() {
  portENTER_CRITICAL_ISR(&muxEnc);
  if (digitalRead(ENC_L_B))
    encCountL++;
  else
    encCountL--;
  portEXIT_CRITICAL_ISR(&muxEnc);
}

void IRAM_ATTR isr_EncR() {
  portENTER_CRITICAL_ISR(&muxEnc);
  if (digitalRead(ENC_R_B))
    encCountR++;
  else
    encCountR--;
  portEXIT_CRITICAL_ISR(&muxEnc);
}

// Đọc MPU6050 đơn giản, trả về true nếu thành công
bool readMPU(int16_t *accX, int16_t *accZ, int16_t *gyrY) {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  if (Wire.endTransmission(false) != 0) {
    return false; // I2C lỗi
  }
```

```
  uint8_t count =
      Wire.requestFrom((uint8_t)MPU_ADDR, (uint8_t)14, (uint8_t)true);
  if (count != 14) {
    // Clear buffer nếu có
    while (Wire.available())
      Wire.read();
    return false;
  }

  *accX = Wire.read() << 8 | Wire.read();
  Wire.read();
  Wire.read(); // accY - bỏ qua
  *accZ = Wire.read() << 8 | Wire.read();
  Wire.read();
  Wire.read(); // Temp - bỏ qua
  Wire.read();
  Wire.read(); // gyrX - bỏ qua
  *gyrY = Wire.read() << 8 | Wire.read();
  Wire.read();
  Wire.read(); // gyrZ - bỏ qua

  return true;
}

void initMPU6050() {
  Wire.begin(21, 22);    // SDA=21, SCL=22 (mặc định ESP32)
  Wire.setClock(400000); // 400kHz I2C

  // Wake up MPU6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);
```

```
  Wire.write(0x00);
  Wire.endTransmission(true);

  // Gyro config: ±250°/s
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1B);
  Wire.write(0x00);
  Wire.endTransmission(true);

  // Accel config: ±2g
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1C);
  Wire.write(0x00);
  Wire.endTransmission(true);

  Serial.println("[SENSOR] MPU6050 OK");
}

void TaskSensor(void *pvParameters) {
  initMPU6050();

  // Encoder pins
  pinMode(ENC_L_A, INPUT);
  pinMode(ENC_L_B, INPUT);
  pinMode(ENC_R_A, INPUT);
  pinMode(ENC_R_B, INPUT);

  attachInterrupt(ENC_L_A, isr_EncL, RISING);
  attachInterrupt(ENC_R_A, isr_EncR, RISING);

  float filtered_theta = 0.0;
  long prevPosL = 0, prevPosR = 0;
```

```
int errorCount = 0;

// Warm-up: Đọc góc ban đầu
vTaskDelay(pdMS_TO_TICKS(200));
int16_t ax, az, gy;
if (readMPU(&ax, &az, &gy)) {
  filtered_theta = -atan2((float)ax, (float)az);
  Serial.printf("[SENSOR] Init: %.1f deg\n", filtered_theta * 180.0 / PI);
} else {
  Serial.println("[SENSOR] MPU read FAILED!");
}

TickType_t xLastWakeTime = xTaskGetTickCount();

for (;;) {
  vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(DT_MS));

  int16_t accX, accZ, gyrY;

  if (readMPU(&accX, &accZ, &gyrY)) {
    errorCount = 0;

    float thetaAcc = -atan2((float)accX, (float)accZ);
    float thetaDot = (gyrY / 131.0f) * (PI / 180.0f); // Đã sửa dấu

    // Complementary filter
    filtered_theta = ALPHA * (filtered_theta + thetaDot * DT_S) +
                     (1.0f - ALPHA) * thetaAcc;

    // Đọc encoder
    portENTER_CRITICAL(&muxEnc);
    long currL = encCountL;
```

```
      long currR = encCountR;
      portEXIT_CRITICAL(&muxEnc);

      float x_meters = ((currL + currR) / 2.0f) * METERS_PER_COUNT;
      float x_dot = (((currL - prevPosL) + (currR - prevPosR)) / 2.0f *
                  METERS_PER_COUNT) /
                 DT_S;

      prevPosL = currL;
      prevPosR = currR;

      // Cập nhật state
      if (xSemaphoreTake(xStateMutex, pdMS_TO_TICKS(2))) {
        robotState.theta = filtered_theta;
        robotState.theta_dot = thetaDot;
        robotState.x = x_meters;
        robotState.x_dot = x_dot;
        xSemaphoreGive(xStateMutex);
      }
    } else {
      // Lỗi I2C - chỉ log, không reset (để tránh treo)
      errorCount++;
      if (errorCount % 100 == 0) {
        Serial.printf("[SENSOR] I2C err x%d\n", errorCount);
      }
    }
  }
}
```

----------4. PostureBlock.h----------

```
#ifndef POSTURE_BLOCK_H
```

```
#define POSTURE_BLOCK_H

#include "SensorBlock.h"
#include <Arduino.h>
#include <ESP32Servo.h>

#define PIN_SERVO_1 18 // Servo Phải
#define PIN_SERVO_2 19 // Servo Trái

#define MIN_US 500
#define MAX_US 2500
#define MAX_ANGLE 260

#define FIXED_ANGLE 155

extern int servoAngle;       // Góc hiện tại
extern int targetServoAngle; // Góc đích (set từ Serial)

void setupPosture();
void TaskPosture(void *pvParameters);

#endif
```

----------5.PostureBlock.cpp----------

```
#include "PostureBlock.h"

Servo servo1;
Servo servo2;

int servoAngle = 155;       // Góc hiện tại
int targetServoAngle = 155; // Góc đích (từ Serial)
```

```
void setServoAngle(Servo &sv, int angle) {
  if (angle < 0)
    angle = 0;
  if (angle > MAX_ANGLE)
    angle = MAX_ANGLE;
  int us = map(angle, 0, MAX_ANGLE, MIN_US, MAX_US);
  sv.writeMicroseconds(us);
}

void setupPosture() {
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);

  servo1.setPeriodHertz(50);
  servo1.attach(PIN_SERVO_1, MIN_US, MAX_US);

  servo2.setPeriodHertz(50);
  servo2.attach(PIN_SERVO_2, MIN_US, MAX_US);

  int angle1 = servoAngle;
  int angle2 = MAX_ANGLE - servoAngle;

  setServoAngle(servo1, angle1);
  setServoAngle(servo2, angle2);
}

void TaskPosture(void *pvParameters) {
  TickType_t xLastWakeTime = xTaskGetTickCount();

  for (;;) {
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(50)); // 50ms = 20Hz
```

```
    // =====thay đổi từ từ 1° mỗi chu kỳ =====
    if (servoAngle < targetServoAngle) {
      servoAngle++;
    } else if (servoAngle > targetServoAngle) {
      servoAngle--;
    }
    // ==============================================

    int angle1 = servoAngle;
    int angle2 = MAX_ANGLE - servoAngle;
    setServoAngle(servo1, angle1);
    setServoAngle(servo2, angle2);
  }
}
```

----------6. ControlBlock.h----------

```
#ifndef CONTROL_BLOCK_H
#define CONTROL_BLOCK_H

#include <Arduino.h>
#include <freertos/FreeRTOS.h>
#include <freertos/queue.h>
#include "SensorBlock.h"

struct MotorCommand {
  float voltageL;
  float voltageR;
};

extern QueueHandle_t queueMotor;
```

```
void TaskController(void *pvParameters);

#endif
```

----------7. ControlBlock.cpp----------

```
#include "ControlBlock.h"
#include "PostureBlock.h"

// --- THÔNG SỐ KHÔNG ĐỔI ---
const float V_MIN_KICK = 2.0;
const float V_MAX = 10.0;

// ====== FUZZY LOGIC ======
// Tam giác: low(115), mid(135), high(155)
void fuzzyMembership(int angle, float &wLow, float &wMid, float &wHigh) {
  wLow = wMid = wHigh = 0.0f;

  if (angle <= 115) {
    wLow = 1.0f;
  } else if (angle <= 135) {
    wLow = (135.0f - angle) / 20.0f;
    wMid = (angle - 115.0f) / 20.0f;
  } else if (angle <= 155) {
    wMid = (155.0f - angle) / 20.0f;
    wHigh = (angle - 135.0f) / 20.0f;
  } else {
    wHigh = 1.0f;
  }
}

// output = wLow*low + wMid*mid + wHigh*high
```

```
float fuzzyOutput(float wLow, float wMid, float wHigh, float low, float mid,
                  float high) {
  return wLow * low + wMid * mid + wHigh * high;
}
// ============================

void TaskController(void *pvParameters) {
  RobotState state;
  MotorCommand cmd;

  TickType_t xLastWakeTime = xTaskGetTickCount();
  for (;;) {
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10)); // 100Hz

    // 1. Lấy dữ liệu an toàn
    if (xSemaphoreTake(xStateMutex, 5) == pdTRUE) {
      state = robotState;
      xSemaphoreGive(xStateMutex);
    } else {
      continue;
    }

    // 2. FUZZY: Tính TARGET_ANGLE và K
    float wLow, wMid, wHigh;
    fuzzyMembership(servoAngle, wLow, wMid, wHigh);

    float target = fuzzyOutput(wLow, wMid, wHigh, 11.0, 12.5, 13.7);
    float K_THETA_F = fuzzyOutput(wLow, wMid, wHigh, 10.0, 20.0, 50.0);
    float K_THETA_DOT_F = fuzzyOutput(wLow, wMid, wHigh, 1.0, 1.0, 1.5);
    float K_X_F = fuzzyOutput(wLow, wMid, wHigh, 0.0, 0.0, 0.0);
    float K_X_DOT_F = fuzzyOutput(wLow, wMid, wHigh, 0.0, 0.0, 0.0);
```

```
// 3. TÍNH TOÁN LQR
float theta_error = state.theta - (target * PI / 180.0);

float u =
    -(K_THETA_F * theta_error + K_THETA_DOT_F * (state.theta_dot + 0.03)
+
      K_X_F * state.x + K_X_DOT_F * state.x_dot);

// 4. BÙ MA SÁT
float u_out = 0;
if (u > 0)
  u_out = u + V_MIN_KICK;
if (u < 0)
  u_out = u - V_MIN_KICK;

// 5. Saturation
if (u_out > V_MAX)
  u_out = V_MAX;
if (u_out < -V_MAX)
  u_out = -V_MAX;

// Deadzone
if (abs(theta_error) < 0.005)
  u_out = 0;

// Lưu u_out
if (xSemaphoreTake(xStateMutex, 2) == pdTRUE) {
  robotState.u_out = u_out;
  xSemaphoreGive(xStateMutex);
}

// Xuất lệnh
```

```
    cmd.voltageL = u_out;
    cmd.voltageR = u_out;
    xQueueOverwrite(queueMotor, &cmd);
  }
}
```

----------8. MotorBlock.h----------

```
#ifndef MOTOR_BLOCK_H
#define MOTOR_BLOCK_H

#include <Arduino.h>
#include "ControlBlock.h"

#define PIN_RPWM_L 25
#define PIN_LPWM_L 26
#define PIN_RPWM_R 33
#define PIN_LPWM_R 32

void TaskMotor(void *pvParameters);

#endif
```

----------9. MotorBlock.cpp----------

```
#include "MotorBlock.h"

#define MAX_VOLTAGE 12.0
#define DEADZONE    20
#define PWM_FREQ    5000
#define PWM_RES     8
```

```
void setMot(int pin_rpwm, int pin_lpwm, float v) {
  if (v > MAX_VOLTAGE) v = MAX_VOLTAGE;
  if (v < -MAX_VOLTAGE) v = -MAX_VOLTAGE;

  int pwm = (int)(abs(v) / MAX_VOLTAGE * 255.0);

  if (pwm < DEADZONE) {
    ledcWrite(pin_rpwm, 0);
    ledcWrite(pin_lpwm, 0);
    return;
  }

  if (v > 0) {
    ledcWrite(pin_rpwm, pwm);
    ledcWrite(pin_lpwm, 0);
  }
  else {
    ledcWrite(pin_rpwm, 0);
    ledcWrite(pin_lpwm, pwm);
  }
}

void TaskMotor(void *pvParameters) {
  // ESP32 Core v3.0+
  ledcAttach(PIN_RPWM_L, PWM_FREQ, PWM_RES);
  ledcAttach(PIN_LPWM_L, PWM_FREQ, PWM_RES);
  ledcAttach(PIN_RPWM_R, PWM_FREQ, PWM_RES);
  ledcAttach(PIN_LPWM_R, PWM_FREQ, PWM_RES);

  MotorCommand cmd;

  for (;;) {
```

```
    if (xQueueReceive(queueMotor, &cmd, portMAX_DELAY)) {
      setMot(PIN_RPWM_L, PIN_LPWM_L, cmd.voltageL);
      setMot(PIN_RPWM_R, PIN_LPWM_R, cmd.voltageR);
    }
  }
}
```

----------10. DebugBlock.h----------

```
#ifndef DEBUG_BLOCK_H
#define DEBUG_BLOCK_H

#include "SensorBlock.h"

void TaskDebug(void *pvParameters);

#endif
```

----------11. DebugBlock.cpp----------

```
#include "DebugBlock.h"
#include "PostureBlock.h"

void TaskDebug(void *pvParameters) {
  RobotState s;
  for (;;) {
    // ===== Đọc góc từ Serial =====
    if (Serial.available()) {
      int newAngle = Serial.parseInt();
      if (newAngle >= 0 && newAngle <= 260) {
        targetServoAngle = newAngle;
        Serial.printf("[POSTURE] Target angle: %d (smooth)\n",
                      targetServoAngle);
```

```
    }
    while (Serial.available())
      Serial.read();
  }
  // ==============================

  if (xSemaphoreTake(xStateMutex, 10)) {
    s = robotState;
    xSemaphoreGive(xStateMutex);
  }
  // In ra Serial Plotter
  Serial.print("Theta:");
  Serial.print(s.theta * 180.0 / PI);
  Serial.print(" ,ThetadOT:");
  Serial.print(s.theta_dot);
  //Serial.print(" ,X_cm:");
  //Serial.print(s.x * 100.0, 1);
  //Serial.print(" ,Servo:");
  //Serial.print(servoAngle);
  Serial.println();



  vTaskDelay(50 / portTICK_PERIOD_MS);
 }
}
```

Appendix 2: MATLAB Script for Center of Mass (CoM) Calculation of the Self-Balancing Robot Body

```
clc; clear; close all;
DH = @(th, al, a, d) [cos(th), -sin(th), 0, a; ...
                      sin(th)*cos(al), cos(th)*cos(al), -sin(al), -d*sin(al); ...
                      sin(th)*sin(al), cos(th)*sin(al), cos(al), d*cos(al); ...
                      0, 0, 0, 1];
```

```
m = [0.20503;   % m1: Cẳng chân
     0.02573;   % m2: Đùi dưới
     0.02923;   % m3: Đùi trên
     1.07152];  % m4: Thân xe
l1 = 0.1155;
l2 = 0.08;
l3 = 0.0954;
l4 = 0.07422;
a = 0.04377;
lc = [0.05928; % lc1
      0.0459;  % lc2
      0.0477;  % lc3
      0.00438]; % lc4
goc_dk= 115;
goc_do=goc_dk-37.65;
goc_rad = deg2rad(goc_do);
l25 = sqrt(l2^2 + l4^2 - 2*l2*l4*cos(goc_rad));
goc21 = acos((l25^2 + a^2 - l3^2)/(2*l25*a));
goc22 = acos((l25^2 + l2^2 - l4^2)/(2*l25*l2));
goc23 = pi - goc21 - goc22;
h = sqrt(l2^2 + (l1 - a)^2 - 2*l2*(l1 - a)*cos(goc23))
%---------------------goc_dk ===> Lc ---------------------------------
theta_1 = acos( (h^2 + (l1 - a)^2 - l2^2) / (2*h*(l1 - a)) );
theta_2 = goc21 + goc22;
theta_3 = pi - acos((l3^2 + a^2 - l25^2)/(2*l3*a));
theta_4 = pi - goc_rad;
T_rel = cell(4, 1); % Chỉ cần 3 ma trận quan hệ (2vs1, 3vs2, 4vs3)
T_rel{1} = DH(theta_1, 0, l1, 0);
% Ma trận chuyển từ Link 2 sang Link 3
T_rel{2} = DH(theta_2, 0, l2, 0);
T_rel{3} = DH(theta_3, 0, l3, 0);
T_global = cell(4, 1);
```

```
T_global{1} = eye(4);
T_global{2} = T_rel{1};
T_global{3} = T_global{2} * T_rel{2};
T_global{4} = T_global{3} * T_rel{3};
numerator_sum_vec = [0; 0; 0; 0];
denominator_sum_m = 0;
fprintf('--- Chi tiết vị trí CoM (So với hệ tọa độ Khâu 1) ---\n');
for i = 1:4
    P_local = [lc(i); 0; 0; 1];
    P_global = T_global{i} * P_local;
    fprintf('Link %d: Mass=%.2f, Pos=[%.4f, %.4f]\n', ...
            i, m(i), P_global(1), P_global(2));
    numerator_sum_vec = numerator_sum_vec + (m(i) * P_global);
    denominator_sum_m = denominator_sum_m + m(i);
end
wPc = numerator_sum_vec / denominator_sum_m;
xc = wPc(1);
zc = wPc(2);
lc_equivalent = sqrt(xc^2 + zc^2);
fprintf('KẾT QUẢ (HỆ QUY CHIẾU: LINK 1)\n');
fprintf('Tọa độ X (xc): %.4f m\n', xc);
fprintf('Tọa độ Z (zc): %.4f m\n', zc);
fprintf('Khoảng cách từ gốc Link 1 đến CoM tổng: %.4f m\n', lc_equivalent);
```

Appendix 3: MATLAB Script for Calculating the LQR Gain Matrix (K) for the Self-Balancing Robot System

```
clc;
M = 1.8915;
m = 0.055;
l = 0.0715;
g = 9.81;
r1 = 1;
```

```
thetadot_init = 0 ;
theta_init = 0.1;
xdot_init = 0;
x_init = 0;
A = [ 0,            1, 0, 0;
    ((M + m)*g)/(M*l), 0, 0, 0;
     0,             0, 0, 1;
    -(m*g)/M,       0, 0, 0];
B = [ 0;
    -1/(M*l);
     0;
     1/M];
Q = [ 1 0 0 0; 0 1 0 0; 0 0 100 0; 0 0 0 1]
R = r1;
k = lqr(A,B,Q,R)
```